VS CODE

CHEAT SHEETS

A Keyboard-First Reference for Visual Studio Code

Command Palette · Settings · Profiles · Multi-Cursor · Navigation · Search & Replace · Snippets · IntelliSense · Refactoring · Formatting · Terminal · Debugging · launch.json · Tasks · Testing · Git · Merge Editor · Extensions · Workspaces · Dev Containers · Copilot · Agents & MCP

29 REFERENCE SHEETS

CONTENTS

SETUP & INTERFACE

01 Introduction & Installation
02 The Workbench Layout
03 Command Palette & Quick Open
04 Settings
05 Profiles, Themes & Keybindings

EDITING & NAVIGATION

06 Multi-Cursor & Selection
07 Line Editing Shortcuts
08 Code Navigation
09 Search & Replace
10 Snippets & Emmet
11 Split Editors, Folding & View Options

CODE INTELLIGENCE

12 IntelliSense & Language Servers
13 Refactoring & Code Actions
14 Formatting & Linting

RUNNING, DEBUGGING & TESTING

15 Integrated Terminal
16 Debugging Basics
17 launch.json Configurations
18 Tasks
19 The Testing View

SOURCE CONTROL

20 Git in VS Code
21 Diff & Merge Editor
22 GitHub Integration

EXTENSIONS, WORKSPACES & REMOTE

23 Managing Extensions
24 Workspaces & Multi-Root
25 Remote Development

AI IN THE EDITOR

26 Copilot Completions & Chat
27 Agent Mode, MCP & Other AI Tools

QUICK REFERENCE

28 Quick Reference — Keyboard Shortcuts
29 Quick Reference — Settings, CLI & Files

VSC

Table of Contents

29 sheets · syntax · examples · notes

SETUP & INTERFACE 01 – 05

- 01 Introduction & Installat…
- 02 The Workbench Layout
- 03 Command Palette & Q…
- 04 Settings
- 05 Profiles, Themes & Key…

EDITING & NAVIGATION 06 – 11

- 06 Multi-Cursor & Selecti…
- 07 Line Editing Shortcuts
- 08 Code Navigation
- 09 Search & Replace
- 10 Snippets & Emmet
- 11 Split Editors, Folding &…

CODE INTELLIGENCE 12 – 14

- 12 IntelliSense & Languag…
- 13 Refactoring & Code Ac…
- 14 Formatting & Linting

RUNNING, DEBUGGING & TESTING 15 – 19

- 15 Integrated Terminal
- 16 Debugging Basics
- 17 launch.json Configurati…
- 18 Tasks
- 19 The Testing View

SOURCE CONTROL 20 – 22

- 20 Git in VS Code
- 21 Diff & Merge Editor
- 22 GitHub Integration

EXTENSIONS, WORKSPACES & REMOTE 23 – 25

- 23 Managing Extensions
- 24 Workspaces & Multi-R…
- 25 Remote Development

AI IN THE EDITOR 26 – 27

- 26 Copilot Completions &…
- 27 Agent Mode, MCP & Ot…

QUICK REFERENCE 28 – 29

- 28 Quick Reference — Ke…
- 29 Quick Reference — Set…

Sheet 01

Introduction & Installation

Stable vs Insiders · code CLI · code . · code -d · code -g file:line · VS Code forks

A FREE EDITOR THAT ACTS LIKE AN IDE

Visual Studio Code is a free, open-source-based code editor from Microsoft that runs on macOS, Windows and Linux. Out of the box it edits, searches and runs Git; **extensions** and **language servers** add IntelliSense, debugging and testing for almost every language. Install **Stable** for daily work (monthly feature releases); **Insiders** ships nightly. The `code` command opens files and folders from the terminal.

NOTES

- Download from `code.visualstudio.com`, or `brew install --cask visual-studio-code` / `winget install Microsoft.VisualStudioCode`
- macOS: run **Shell Command: Install 'code' command in PATH** from the Command Palette
- Windows / Linux installers add `code` to PATH for you
- Forks such as **Cursor**, **Windsurf** and **VSCodium** share almost everything on these sheets

THE CODE CLI

```
code .                          # open the current folder
code app.py README.md           # open files in the last window
code -n ~/projects/api          # new window
code -r src/                    # reuse the current window
code -g src/app.ts:42:7         # go to line 42, column 7
code -d old.json new.json       # diff two files
code --wait COMMIT_EDITMSG      # block until the tab closes
git config --global core.editor "code --wait"
code --list-extensions          # installed extension IDs
code --version                  # 1.1xx.x  commit  arch
code --disable-extensions .     # troubleshoot without extensions
```

CODE EXPLAINED

- → `code .` opens the folder as the workspace, which is what Explorer, Search and Git all work from
- → `-g` takes `file:line:column`; terminal links in VS Code use the same format
- → `--wait` makes `code` usable as Git's editor for commit messages and rebase lists
- → Use `code-insiders` for the Insiders build; both can be installed side by side

SETTINGS.JSON

- {} `"update.mode": "default"` — check for updates automatically (`"manual"` / `"none"`)
- {} `"window.restoreWindows": "all"` — reopen every window on launch
- {} `"telemetry.telemetryLevel": "off"` — turn off usage data

IN OTHER EDITORS

- **JetBrains** — launcher `idea .` / `pycharm .`; **File → Open** a folder to make it a project
- **Vim / Neovim** — `vim .` or `nvim .` opens a file browser (netrw) on the folder

BUILDS & FORKS

BUILD	WHAT IT IS
Stable	Monthly releases, the one to use
Insiders	Nightly preview, separate settings
VSCodium	Microsoft-free build, Open VSX marketplace
Cursor / Windsurf	AI-focused forks, import VS Code settings
vscode.dev	Browser version, no terminal

WHERE SETTINGS LIVE

FILE	HOLDS
settings.json	User or workspace settings
keybindings.json	Your shortcut overrides
.vscode/	Per-project settings, launch, tasks
snippets/*.json	User snippets
extensions/	Installed extensions (~/.vscode)

⚠ COMMON MISTAKES

- Opening single files instead of the project folder, so search, Git and IntelliSense see nothing else
- Forgetting the macOS **Install 'code' command** step and wondering why `code` is not found
- Mixing Stable and Insiders settings: they are separate installs with separate profiles

✓ TIPS & BEST PRACTICES

- Set Git's editor to `code --wait` so commits and interactive rebases open in VS Code
- Launch with `--disable-extensions` to tell an editor bug from an extension bug
- Turn on **Settings Sync** right after installing so a new machine is one sign-in away

QUICK SUMMARY

- `code .` — open this folder
- `code -g f:l` — jump to a line
- `code -d a b` — diff two files
- `--wait` — use VS Code as Git's editor

The Workbench Layout

FIVE AREAS AROUND THE EDITOR

The window is the **workbench**. The **Activity Bar** picks a view (Explorer, Search, Source Control, Run and Debug, Extensions); the **Primary Side Bar** shows it. The **Secondary Side Bar** on the other side holds Chat and anything you drag there. The **Panel** below the editor holds the terminal, Problems, Output and Debug Console. The **Status Bar** shows branch, errors, language and line endings.

NOTES

- Every view can be dragged to another area or the Status Bar
- **View → Appearance** toggles each area and moves the Panel
- Right-click the Activity Bar to hide views you never use
- **Zen Mode** hides everything but the editor (Esc Esc to leave)

SET UP A CLEAN LAYOUT

1. Hide what you don't use: right-click the Activity Bar and untick views
2. Drag **Outline** and **Timeline** from Explorer into the Secondary Side Bar
3. Toggle the Side Bar with ⌘B / Ctrl+B and the Panel with ⌘J / Ctrl+J
4. Maximize the Panel with its ^ button when reading long terminal output
5. Use **Customize Layout** (title bar icon) to try positions with a live preview
6. Save the result in a **profile** (sheet 05) so other projects get it too

WHERE TO FIND IT

AREA	TOGGLE / OPEN
Primary Side Bar	⌘B / Ctrl+B
Secondary Side Bar	⌥⌘B / Ctrl+Alt+B
Panel	⌘J / Ctrl+J
Explorer	⇧⌘E / Ctrl+Shift+E
Problems	⇧⌘M / Ctrl+Shift+M
Output	⇧⌘U / Ctrl+Shift+U
Zen Mode	⌘K Z / Ctrl+K Z

SETTINGS.JSON

- {} `"workbench.activityBar.location": "top"` — icons above the Side Bar (`"hidden"` removes them)
- {} `"workbench.sideBar.location": "right"` — keeps code from jumping when it opens
- {} `"workbench.editor.showTabs": "single"` — one tab line (`"none"` hides tabs)
- {} `"zenMode.centerLayout": true` — center the editor in Zen Mode

IN OTHER EDITORS

- **JetBrains** — tool windows: ⌘1 / Alt+1 Project, ⇧⌘F12 / Ctrl+Shift+F12 hides all; Distraction Free Mode
- **Vim / Neovim** — no workbench; windows and tabs, plus a file tree plugin (neo-tree, nvim-tree, NERDTree)

ACTIVITY BAR VIEWS

VIEW	SHORTCUT
Explorer	⇧⌘E / Ctrl+Shift+E
Search	⇧⌘F / Ctrl+Shift+F
Source Control	^⇧G / Ctrl+Shift+G
Run and Debug	⇧⌘D / Ctrl+Shift+D
Extensions	⇧⌘X / Ctrl+Shift+X
Testing	Activity Bar beaker icon

STATUS BAR ITEMS

ITEM	CLICK TO
Branch name	Switch or create a branch
⊗ 2 ⚠ 5	Open Problems
Ln 12, Col 4	Go to Line
Spaces: 2	Change indentation
LF / CRLF	Change line endings
{ } TypeScript	Change language mode

⚠ COMMON MISTAKES

- Closing the Panel when you meant to close one terminal: the terminal keeps running and ⌘J / Ctrl+J brings it back
- Leaving every Activity Bar view visible; clutter makes the shortcuts the only fast path anyway
- Missing errors because the Status Bar was hidden; the error count lives there

✓ TIPS & BEST PRACTICES

- Put the Side Bar on the right so opening it never shifts your code
- Use the Secondary Side Bar for things you want open at the same time as Explorer (Chat, Outline)
- Double-click a tab to maximize its editor group

QUICK SUMMARY

- `⌘B / Ctrl+B` — Side Bar
- `⌘J / Ctrl+J` — Panel
- `⌥⌘B / Ctrl+Alt+B` — Secondary Side Bar
- `⌘K Z` — Zen Mode

Sheet 03

Command Palette & Quick Open

⇧⌘P / Ctrl+Shift+P · ⌘P / Ctrl+P · > @ : # ? prefixes · recent files · keybinding lookup

ONE BOX TO REACH EVERYTHING

Quick Open (⌘P / Ctrl+P) finds files by fuzzy name. Typing a **prefix** turns the same box into other tools: > runs a command (the **Command Palette**, ⇧⌘P / Ctrl+Shift+P), @ jumps to a symbol in this file, # searches symbols across the workspace, : goes to a line and ? lists every prefix. Every command shows its shortcut, so the palette is also how you learn them.

NOTES

- Fuzzy matching: `usrctl` finds `UserController.ts`
- Recently used files and commands float to the top
- ⌘Enter / Ctrl+Enter opens the result to the side
- Press ⌘P / Ctrl+P twice to toggle between the last two files

PREFIXES IN THE QUICK OPEN BOX

```
app.ts              # file by name (fuzzy)
app.ts:42           # file, then line 42
>format doc         # run a command
>Preferences: Open User Settings (JSON)
@render             # symbol in this file
@:                  # symbols grouped by kind
#UserService        # symbol anywhere in the workspace
:120                # go to line 120
edt (tab)           # open editors list
term                # pick a terminal (prefix "term ")
task                # run a task (prefix "task ")
debug               # start a launch configuration
?                   # show all prefixes
```

CODE EXPLAINED

→ File, command and symbol search all live in one widget; the first character decides the mode

→ `file:line` jumps straight to a line, handy when pasting from a stack trace

→ `@:` groups symbols as Classes, Methods, Properties so long files are easy to scan

→ The word prefixes (`term` , `task` , `debug`) need a space after them

SETTINGS.JSON

{} `"workbench.commandPalette.history": 50` — how many recent commands to keep

{} `"workbench.quickOpen.closeOnFocusLost": false` — keep the box open when you click away

{} `"search.quickOpen.includeSymbols": true` — mix workspace symbols into file results

IN OTHER EDITORS

- **JetBrains** — Search Everywhere: double ⇧ / double Shift; Find Action ⇧⌘A / Ctrl+Shift+A
- **Vim / Neovim** — `:` command line; Telescope / fzf-lua (`find_files` , `commands` , `lsp_document_symbols`)

PREFIX REFERENCE

PREFIX	MODE	DIRECT SHORTCUT
(none)	Go to File	⌘P / Ctrl+P
>	Command Palette	⇧⌘P / Ctrl+Shift+P · F1
@	Symbol in file	⇧⌘O / Ctrl+Shift+O
#	Symbol in workspace	⌘T / Ctrl+T
:	Go to Line	^G / Ctrl+G
?	Help	—

LEARN THE SHORTCUTS

WANT TO	DO
See a command's key	Read it on the right of the palette row
Change a key	Gear icon on the palette row
Search all keys	⌘K ⌘S / Ctrl+K Ctrl+S
Find by key	Keyboard Shortcuts → keyboard icon, press keys
Printable list	Help → Keyboard Shortcuts Reference

⚠ COMMON MISTAKES

- Clicking through menus for things the palette finds in two keystrokes
- Typing `>` in ⌘P / Ctrl+P and forgetting that ⇧⌘P / Ctrl+Shift+P already adds it
- Searching for a function with `@` when it lives in another file; use `#` for the workspace

✓ TIPS & BEST PRACTICES

- Use ⌘P / Ctrl+P then `@` to land in a file and a symbol in one go
- F1 also opens the Command Palette and is easier to hit on some keyboards
- Bind your top three palette commands to keys from the gear icon

QUICK SUMMARY

- `⌘P / Ctrl+P` — files
- `⇧⌘P / Ctrl+Shift+P` — commands
- `@` / `#` — symbols here / everywhere
- `:` — line

Settings

Settings editor vs `settings.json` · User / Workspace / Folder · `[python]` overrides · `@modified` · Settings Sync

LAYERS OF JSON, MOST SPECIFIC WINS

Every option is a key in a **settings.json** file. **User** settings apply everywhere; **Workspace** settings live in `.vscode/settings.json` and override them for one project; in a multi-root workspace each **Folder** can override again. A `"[language]"` block overrides by file type. The **Settings editor** (⌘, / Ctrl+,) is a searchable UI over the same files; the {} icon opens the JSON.

NOTES

- Workspace settings are committed with the repo; keep personal taste in User
- Search filters: `@modified`, `@ext:ms-python.python`, `@lang:python`, `@tag:experimental`
- Hover a setting's gear → **Copy Setting ID** for the JSON key
- **Settings Sync** (Accounts menu) syncs settings, keys, snippets, extensions, profiles

SETTINGS.JSON (JSON WITH COMMENTS)

```jsonc
{
  // User: ~/Library/Application Support/Code/User/settings.json
  "editor.fontSize": 14,
  "editor.tabSize": 2,
  "editor.rulers": [80, 120],
  "files.autoSave": "onFocusChange",
  "files.trimTrailingWhitespace": true,
  "files.insertFinalNewline": true,

  // Language-specific override
  "[python]": {
    "editor.tabSize": 4,
    "editor.formatOnSave": true
  },
  "[markdown]": { "editor.wordWrap": "on" }
}
```

CODE EXPLAINED

→ Settings files are **JSONC**: comments and trailing commas are allowed

→ Windows path: `%APPDATA%\Code\User\settings.json`; Linux: `~/.config/Code/User/`

→ `"[python]"` applies only to Python files and beats both User and Workspace values

→ Unknown or misspelled keys are greyed out with a warning squiggle

SETTINGS.JSON

{} `"workbench.settings.editor": "json"` — open settings straight to JSON

{} `"files.autoSave": "afterDelay"` + `"files.autoSaveDelay": 1000`

{} `"editor.rulers": [80]` — vertical guide lines at those columns

IN OTHER EDITORS

- **JetBrains** — Settings ⌘, / Ctrl+Alt+S, stored as XML in `.idea/` (project) and the IDE config dir; Settings Sync via JetBrains account
- **Vim / Neovim** — `~/.vimrc` or `~/.config/nvim/init.lua`; `:set tabstop=2`; `ftplugin/python.lua` for per-language settings

SCOPES (LOWEST → HIGHEST)

SCOPE	FILE	COMMIT?
`Default`	built in	—
`User`	User/settings.json	No (sync it)
`Workspace`	.vscode/settings.json	Yes
`Folder`	<folder>/.vscode/settings.json	Yes
`Language`	"[lang]": { … } in any file	Either

OPENING SETTINGS

COMMAND	OPENS
`⌘, / Ctrl+,`	Settings editor
`Open User Settings (JSON)`	User settings.json
`Open Workspace Settings (JSON)`	.vscode/settings.json
`Open Default Settings (JSON)`	Read-only defaults
`Settings Sync: Turn On`	Sign in and sync

⚠ COMMON MISTAKES

- Committing personal preferences (font size, theme) in `.vscode/settings.json` for the whole team
- Editing a setting in the UI under the wrong tab (User vs Workspace) and wondering why it doesn't apply
- Copying JSON with a missing comma; VS Code ignores the whole file until it parses

✓ TIPS & BEST PRACTICES

- Filter by `@modified` to see exactly what you have changed from the defaults
- Put formatter and linter settings in Workspace so every contributor gets them
- Use **Copy Setting ID** instead of guessing key names

QUICK SUMMARY

- `⌘, / Ctrl+,` — Settings editor
- `settings.json` — the real source (JSONC)
- `"[lang]"` — per-language override
- Workspace beats User; Folder beats Workspace

Sheet 05

Profiles, Themes & Keybindings

profiles · export / import profile · color & icon themes · fonts & ligatures · keybindings.json · when clauses

DIFFERENT SETUPS FOR DIFFERENT WORK

A **profile** bundles settings, keybindings, snippets, extensions and UI state. Make one per kind of work (Python, Web, Writing) and switch with a click; a folder remembers its profile. **Color themes** and **file icon themes** are just extensions. Custom shortcuts live in **keybindings.json** as `key`, `command` and an optional `when` clause that limits where they fire.

NOTES

- Profiles: gear icon → **Profiles**, or Command Palette **Profiles: New Profile**
- Start a profile from a template (Python, Data Science, Angular…) or copy an existing one
- Export a profile to a GitHub gist or file and share the link
- Theme picker: ⌘K ⌘T / Ctrl+K Ctrl+T (arrow keys preview live)

EXTENSION PICK

- **Material Icon Theme** (Philipp Kief) — file and folder icons by type, easier to scan than the default
- **GitHub Theme** or **One Dark Pro** — popular color themes with light and dark variants

KEYBINDINGS.JSON

```
[
  // Move the terminal toggle, only when the editor has focus
  {
    "key": "ctrl+t",
    "command": "workbench.action.terminal.toggleTerminal",
    "when": "editorTextFocus"
  },
  // Run a palette command with arguments
  {
    "key": "cmd+k cmd+m",
    "command": "workbench.action.editor.changeLanguageMode"
  },
  // Remove a default binding (leading "-")
  { "key": "cmd+k cmd+w", "command": "-workbench.action.closeAllEditors" }
]
```

CODE EXPLAINED

→ Open it from Keyboard Shortcuts (⌘K ⌘S / Ctrl+K Ctrl+S) → **Open Keyboard Shortcuts (JSON)** icon

→ `when` uses context keys like `editorTextFocus`, `terminalFocus`, `resourceExtname == .py`

→ Two-step chords (`cmd+k cmd+m`) are pressed in sequence

→ A `-` before the command removes a default binding instead of adding one

SETTINGS.JSON

{} `"workbench.colorTheme": "Default Dark Modern"`

{} `"window.autoDetectColorScheme": true` — follow the OS light/dark mode

{} `"editor.fontFamily": "'JetBrains Mono', Menlo, monospace"` + `"editor.fontLigatures": true`

IN OTHER EDITORS

- **JetBrains** — Keymap settings and Color Scheme per IDE; no profiles, but separate IDEs fill that role
- **Vim / Neovim** — `:colorscheme`; `vim.keymap.set("n", "<leader>t", …)` in init.lua; `autocmd` for context

PROFILE CONTENTS

PART	SHARED OR PER PROFILE
Settings	Per profile
Keybindings	Per profile
Snippets	Per profile
Extensions	Per profile
UI state / layout	Per profile
Tasks	Per profile

COMMON WHEN CLAUSES

CONTEXT KEY	TRUE WHEN
editorTextFocus	Cursor is in an editor
terminalFocus	Terminal has focus
editorLangId == python	Current file is Python
inDebugMode	A debug session runs
sideBarVisible	Side Bar is open

⚠ COMMON MISTAKES

- Installing every extension in one profile, then living with a slow editor for every project
- Adding a keybinding without a `when` clause and breaking the same key in the terminal
- Editing `keybindings.json` by hand without checking conflicts in the Keyboard Shortcuts editor

✓ TIPS & BEST PRACTICES

- Keep a minimal **Default** profile and add language profiles on top
- Use the **Record Keys** button in Keyboard Shortcuts to find what a key already does
- Sync profiles with Settings Sync so they follow you to new machines

QUICK SUMMARY

- profile — settings + keys + extensions + layout
- `⌘K ⌘T / Ctrl+K Ctrl+T` — pick a theme
- `keybindings.json` — `key` · `command` · `when`
- `-command` — remove a default

Multi-Cursor & Selection

EDIT MANY PLACES AT ONCE

VS Code supports **multiple cursors**: every cursor types, deletes and pastes at the same time. Add them by clicking, by selecting the **next match** of the current word, or by dragging a **column (box) selection**. **Expand selection** grows the selection by syntax (word → string → argument list → statement → block), which is faster than shift-arrowing.

NOTES

- Esc collapses back to one cursor
- Pasting N lines into N cursors puts one line at each cursor
- Matches respect the Find widget's case and whole-word toggles
- Rename Symbol (F2) is safer than multi-cursor for code identifiers

TYPICAL MOVES

```
# Rename every "user" in this block
⌘D ⌘D ⌘D       / Ctrl+D ×3     # add next 3 matches
⌘K ⌘D          / Ctrl+K Ctrl+D # skip this match, take the next
⇧⌘L            / Ctrl+Shift+L  # select ALL matches

# Add a comma to the end of 10 lines
select lines → ⇧⌥I / Shift+Alt+I   # a cursor at each line end

# Column edit
⇧⌥ + drag      / Shift+Alt + drag
⌥⌘↓            / Ctrl+Alt+↓        # cursor on the line below

# Grow and shrink by syntax
^⇧⌘→ / Shift+Alt+→   ^⇧⌘← / Shift+Alt+←
⌘U             / Ctrl+U        # undo last cursor
```

CODE EXPLAINED

- → ⌘D / Ctrl+D with nothing selected selects the word under the cursor first
- → **Select All Occurrences** is the fast version, but check the match count in the Find widget first
- → Cursors at line ends (⇧⌥I / Shift+Alt+I) is the quickest way to edit a column of lines
- → On Linux, Ctrl+Alt+↓ may be taken by the desktop; rebind if it does nothing

SETTINGS.JSON

- {} `"editor.multiCursorModifier": "ctrlCmd"` — make ⌘/Ctrl+Click add cursors (⌥/Alt+Click then opens links)
- {} `"editor.columnSelection": true` — plain drags make box selections
- {} `"editor.multiCursorPaste": "spread"` — one pasted line per cursor

IN OTHER EDITORS

- **JetBrains** — add next occurrence ^G / Alt+J, select all ^⌘G / Ctrl+Alt+Shift+J, column mode ⇧⌘8 / Alt+Shift+Insert, extend selection ⌥↑ / Ctrl+W
- **Vim / Neovim** — `*` then `cgn` and `.` to repeat; Ctrl+V block mode + `I` / `A`; vim-visual-multi plugin for real cursors

ADDING CURSORS

ACTION	MACOS	WINDOWS / LINUX
Add cursor	⌥Click	Alt+Click
Cursor above / below	⌥⌘↑ / ⌥⌘↓	Ctrl+Alt+↑ / ↓
Add next match	⌘D	Ctrl+D
Skip match	⌘K ⌘D	Ctrl+K Ctrl+D
All matches	⇧⌘L	Ctrl+Shift+L
Change all occurrences	⌘F2	Ctrl+F2
Cursors at line ends	⇧⌥I	Shift+Alt+I

SELECTING

ACTION	MACOS	WINDOWS / LINUX
Select line	⌘L	Ctrl+L
Expand selection	^⇧⌘→	Shift+Alt+→
Shrink selection	^⇧⌘←	Shift+Alt+←
Column select	⇧⌥ drag	Shift+Alt drag
Undo cursor	⌘U	Ctrl+U

⚠ COMMON MISTAKES

- Using ⇧⌘L / Ctrl+Shift+L on a common word and changing matches far off-screen
- Renaming a variable with multi-cursor and missing uses in other files; use F2
- Fighting ⌥/Alt+Click because it opens links after changing `multiCursorModifier`

✓ TIPS & BEST PRACTICES

- Use ⌘D / Ctrl+D a few times and watch each match before typing
- Expand selection twice to grab a whole string or argument list
- Paste a list into the same number of cursors to fill a column

QUICK SUMMARY

- `⌘D / Ctrl+D` — next match
- `⇧⌘L / Ctrl+Shift+L` — all matches
- `⇧⌥I / Shift+Alt+I` — cursors at line ends
- Expand selection — grow by syntax

Line Editing Shortcuts

move line ⌥↑ · copy line ⇧⌥↓ · delete line · toggle comment ⌘/ · indent / outdent · join lines · transform case

WORK IN WHOLE LINES

Most editing is moving, copying, deleting and commenting **whole lines**. These commands work on the current line with no selection, or on every line the selection touches, so you never need to select a line first. They also work with multiple cursors.

NOTES

- ⌘X / Ctrl+X and ⌘C / Ctrl+C with nothing selected cut or copy the whole line
- Move Line stays inside the file; it re-indents when the language supports it
- Transform to Upper / Lower / Title / Snake / Kebab case are palette commands (no default key)
- Sort Lines Ascending is a palette command too

CODE EXPLAINED

- → Moving lines with ⌥↑ / Alt+↑ is faster than cut-and-paste and keeps the clipboard
- → Comment toggles use the language's syntax (`#` , `//` , `<!-- -->`) automatically
- → Join Lines has a default key on macOS only; bind it on Windows / Linux if you use it
- → All of these apply to every line under every cursor

RESHAPING A BLOCK

```
⌥↑ / ⌥↓          Alt+↑ / Alt+↓          # move line(s) up / down
⇧⌥↓ / ⇧⌥↑        Shift+Alt+↓ / ↑        # copy line(s) down / up
⇧⌘K              Ctrl+Shift+K           # delete line(s)
⌘↩ / ⇧⌘↩         Ctrl+Enter / +Shift    # new line below / above
⌘/               Ctrl+/                 # toggle line comment
⇧⌥A              Shift+Alt+A            # toggle block comment
⌘] / ⌘[          Ctrl+] / Ctrl+[        # indent / outdent
^J               (palette: Join Lines)  # join with the next line
⇧⌘\              Ctrl+Shift+\           # jump to matching bracket
>Transform to Snake Case               # user_name
>Sort Lines Ascending
```

SETTINGS.JSON

- {} `"editor.emptySelectionClipboard": true` — copy/cut the line when nothing is selected (default)
- {} `"editor.comments.insertSpace": true` — `// text` instead of `//text`
- {} `"editor.autoIndentOnPaste": true` — re-indent pasted code

IN OTHER EDITORS

- **JetBrains** — move line ⌥⇧↑ / Alt+Shift+↑, duplicate ⌘D / Ctrl+D, delete ⌘⌫ / Ctrl+Y, comment ⌘/ / Ctrl+/, join ^⇧J / Ctrl+Shift+J
- **Vim / Neovim** — `:m +1` / `ddp` move, `yyp` copy, `dd` delete, `gcc` comment (built in since Neovim 0.10), `>>` / `<<` , `J` join

LINE COMMANDS

COMMAND	MACOS	WINDOWS / LINUX
Move line up / down	⌥↑ / ⌥↓	Alt+↑ / ↓
Copy line up / down	⇧⌥↑ / ⇧⌥↓	Shift+Alt+↑ / ↓
Delete line	⇧⌘K	Ctrl+Shift+K
Insert line below	⌘↩	Ctrl+Enter
Insert line above	⇧⌘↩	Ctrl+Shift+Enter
Toggle line comment	⌘/	Ctrl+/
Block comment	⇧⌥A	Shift+Alt+A
Indent / outdent	⌘] / ⌘[	Ctrl+] / Ctrl+[

TEXT COMMANDS (PALETTE)

COMMAND	RESULT
Transform to Uppercase	USER_NAME
Transform to Title Case	User Name
Transform to Snake Case	user_name
Transform to Kebab Case	user-name
Sort Lines Ascending	a → z
Delete Duplicate Lines	unique lines
Trim Trailing Whitespace	⌘K ⌘X / Ctrl+K Ctrl+X

⚠ COMMON MISTAKES

- Selecting a line by dragging before copying it; the shortcut already copies the whole line
- Using Copy Line on Linux where Shift+Alt+↓ is taken by the desktop; rebind it
- Block-commenting in languages without block comments (Python) and getting a string literal instead

✓ TIPS & BEST PRACTICES

- Move a function by putting the cursor in it, expanding selection to the block, then ⌥↑ / Alt+↑
- Bind Transform to Snake / Camel Case if you rename keys often
- Use **Sort Lines** on import lists and `.gitignore` files

QUICK SUMMARY

- `⌥↑ / Alt+↑` — move line
- `⇧⌥↓ / Shift+Alt+↓` — copy line
- `⇧⌘K / Ctrl+Shift+K` — delete line
- `⌘/ / Ctrl+/` — comment

Sheet 08

Code Navigation

Go to Definition F12 · Peek ⌥F12 · Find All References · Go to Symbol ⇧⌘O / ⌘T · breadcrumbs · Go Back · Go to Line

MOVE THROUGH CODE BY MEANING

The language server knows what every name refers to. **Go to Definition** (F12, or ⌘/Ctrl+Click) jumps to where a symbol is declared; **Peek** shows it inline without leaving your place. **Find All References** lists every use. **Go Back** (^- / Alt+←) returns along your trail, like a browser's back button.

NOTES

- Navigation needs a language extension; plain-text files only get word search
- **Breadcrumbs** above the editor show the file path and symbol path; click to jump
- The **Outline** view lists the current file's symbols
- **Call Hierarchy** and **Type Hierarchy** are in the right-click menu

FOLLOWING A BUG THROUGH THE CODE

```
F12            / F12              # go to definition
⌘Click         / Ctrl+Click       # same, with the mouse
⌥F12           / Alt+F12          # peek definition (inline)
⌘F12           / Ctrl+F12         # go to implementation
⇧F12           / Shift+F12        # peek references
⇧⌥F12          / Shift+Alt+F12    # find all references (panel)
⇧⌘O            / Ctrl+Shift+O     # symbol in this file
⌘T             / Ctrl+T           # symbol in workspace
^G             / Ctrl+G           # go to line
^-  / ^⇧-      / Alt+← / Alt+→    # back / forward
⇧⌘\            / Ctrl+Shift+\     # matching bracket
F8 / ⇧F8       / F8 / Shift+F8    # next / previous problem
```

CODE EXPLAINED

→ Peek opens an embedded editor: you can edit the definition there and Esc closes it

→ References in the panel stay put while you click through them

→ Back / Forward cross files and remember cursor positions, not just files

→ F8 cycles through errors and warnings in the current file

SETTINGS.JSON

{} `"breadcrumbs.enabled": true` — path and symbol bar above the editor

{} `"editor.gotoLocation.multipleDefinitions": "goto"` — skip the peek when there are several

{} `"workbench.editor.revealIfOpen": true` — reuse an open tab instead of opening a new one

IN OTHER EDITORS

- **JetBrains** — declaration ⌘B / Ctrl+B, implementation ⌥⌘B / Ctrl+Alt+B, find usages ⌥F7 / Alt+F7, file structure ⌘F12 / Ctrl+F12, back ⌘[/ Ctrl+Alt+←
- **Vim / Neovim** — `gd` / LSP definition, `grr` references and `gri` implementation (Neovim 0.11 defaults), `Ctrl+O` / `Ctrl+I` jump list, `%` bracket

GO TO

TARGET	MACOS	WINDOWS / LINUX
Definition	F12	F12
Type definition	palette	palette
Implementation	⌘F12	Ctrl+F12
References	⇧F12	Shift+F12
File	⌘P	Ctrl+P
Symbol in file	⇧⌘O	Ctrl+Shift+O
Symbol anywhere	⌘T	Ctrl+T
Line	^G	Ctrl+G

NAVIGATION VIEWS

VIEW	SHOWS
Breadcrumbs	folder › file › class › method
Outline	Symbols in the current file
References panel	Every use, grouped by file
Call Hierarchy	Who calls this / what it calls
Type Hierarchy	Supertypes and subtypes
Timeline	File history and local saves

⚠ COMMON MISTAKES

- Searching text for a function name and landing in comments and strings; use Find All References
- Opening a new tab for every jump and losing track; use Go Back instead of the tab bar
- Expecting navigation in a file with no language extension installed

✓ TIPS & BEST PRACTICES

- Use Peek (⌥F12 / Alt+F12) to check a signature without losing your place
- Breadcrumbs: ⇧⌘. / Ctrl+Shift+. focuses them so you can pick a sibling symbol by keyboard
- Map the mouse's back/forward buttons to Go Back / Go Forward

QUICK SUMMARY

- `F12` — definition
- `⇧F12 / Shift+F12` — references
- `⇧⌘O` / `⌘T` — symbols
- `^- / Alt+←` — back

Sheet 09

Search & Replace

find / replace in file · Search view ⇧⌘F · regex & capture groups · files to include / exclude · preserve case · Search Editor

FIND IT ONCE, CHANGE IT EVERYWHERE

The **Find widget** (⌘F / Ctrl+F) searches the current file; **Search** (⇧⌘F / Ctrl+Shift+F) searches the whole workspace with ripgrep, honoring `.gitignore`. Both support **regular expressions** with capture groups ($1 in the replacement), **match case** and **whole word**. A **Search Editor** puts results in a full editor tab you can save and re-run.

NOTES

- Toggles: Match Case ⌥⌘C / Alt+C, Whole Word ⌥⌘W / Alt+W, Regex ⌥⌘R / Alt+R
- **Preserve Case** (AB button) keeps `User` → `Account`, `user` → `account`
- Replace previews a diff per file before anything is written
- Click ⋯ in Search to show **files to include / exclude** boxes

EXTENSION PICK

Todo Tree (Gruntfuggly) — collects every `TODO` / `FIXME` in the workspace into a tree, using the same search engine

REGEX REPLACE ACROSS FILES

```
# Search: ⇧⌘H / Ctrl+Shift+H opens Search with Replace
Find:    console\.log\((.*)\);?
Replace: logger.debug($1);
files to include:  src/**/*.ts
files to exclude:  **/*.test.ts, dist

# Rename keys: snake_case → camelCase
Find:    _([a-z])
Replace: \u$1            # \u = uppercase next char

# In-file: ⌥⌘F / Ctrl+H
Find:    ^\s*$\n         # blank lines (regex on)
Replace: (empty)
⌥↩ / Alt+Enter          # select all matches → multi-cursor
```

CODE EXPLAINED

- → `$1`, `$2` insert capture groups; `\u`, `\l`, `\U`, `\L` change case in the replacement
- → Include / exclude boxes take globs, comma-separated; `./src` limits to a folder
- → **Alt+Enter** on Find turns every match into a cursor for hand edits
- → Search skips files in `.gitignore` and `files.exclude` unless you turn that off

SETTINGS.JSON

- {} `"search.exclude": { "/dist": true, "/*.min.js": true }`
- {} `"search.useIgnoreFiles": true` — respect `.gitignore` (default)
- {} `"search.smartCase": true` — case-sensitive only when you type a capital

IN OTHER EDITORS

- **JetBrains** — Find in Files ⇧⌘F / Ctrl+Shift+F, Replace in Files ⇧⌘R / Ctrl+Shift+R, regex `$1` groups the same
- **Vim / Neovim** — `:%s/old/new/gc`, `:vimgrep` / `:grep` + quickfix, `:cdo s/a/b/g` across files; Telescope `live_grep`

FIND & REPLACE KEYS

ACTION	MACOS	WINDOWS / LINUX
Find in file	⌘F	Ctrl+F
Replace in file	⌥⌘F	Ctrl+H
Next / previous	⌘G / ⇧⌘G	F3 / Shift+F3
Search workspace	⇧⌘F	Ctrl+Shift+F
Replace in workspace	⇧⌘H	Ctrl+Shift+H
Select all matches	⌥↩	Alt+Enter

REPLACEMENT TOKENS

TOKEN	INSERTS
$1 … $9	Capture group
$0	Whole match
\u / \l	Upper / lower next char
\U … \E	Uppercase until \E
\n	Newline (regex on)
$$	A literal $

⚠ COMMON MISTAKES

- Replace All across the workspace without reading the preview; build output and vendored code can match too
- Forgetting the regex toggle is on and wondering why `.` matches everything
- Searching inside `node_modules` because `search.exclude` was changed

✓ TIPS & BEST PRACTICES

- Open a **Search Editor** (⇧⌘P → Search Editor: New) for searches you'll re-run
- Use **Preserve Case** for renames that appear as `Foo`, `foo` and `FOO`
- Right-click a folder in Explorer → **Find in Folder** to scope a search

QUICK SUMMARY

- `⌘F / Ctrl+F` — this file
- `⇧⌘F / Ctrl+Shift+F` — workspace
- `$1` — capture group in replace
- include / exclude — glob filters

Snippets & Emmet

user snippets JSON · `prefix` / `body` · `$1` tab stops · `${1:placeholder}` · snippet variables · Emmet abbreviations

TYPE A LITTLE, GET A LOT

Snippets expand a short `prefix` into a template with **tab stops**. Extensions ship snippets for their languages; you add your own with **Snippets: Configure Snippets**, per language (`python.json`), global, or per project (`.vscode/*.code-snippets`). **Emmet** is built in for HTML and CSS: `ul>li*3` + Tab expands to the full markup.

NOTES

- Snippets show in IntelliSense with a square icon
- Tab moves between `$1`, `$2` …; `$0` is the final cursor position
- `${1|a,b,c|}` offers a choice list
- Insert any snippet with the palette **Snippets: Insert Snippet**

EXTENSION PICK

- **Auto Rename Tag** (Jun Han) — renaming an opening HTML/JSX tag renames the closing one
- Language snippet packs (e.g. ES7+ React snippets) add ready-made prefixes

A USER SNIPPET (TYPESCRIPT.JSON)

```
{
  "Test block": {
    "prefix": "desc",
    "description": "describe + it",
    "body": [
      "describe('${1:subject}', () => {",
      "  it('${2:does something}', () => {",
      "    $0",
      "  });",
      "});"
    ]
  },
  "Log value": {
    "prefix": "logv",
    "body": "console.log('${TM_FILENAME}:${TM_LINE_NUMBER}', ${1:value});"
  }
}
```

CODE EXPLAINED

- → `body` is a string or an array of lines; escape `$` as `\\$` when you need it literally
- → `${1:subject}` is a tab stop with default text; typing replaces it
- → Variables like `TM_FILENAME`, `TM_SELECTED_TEXT`, `CURRENT_YEAR` fill in on expansion
- → Emmet: `div.card>h2+p` → nested `div` with an `h2` and a `p`

SETTINGS.JSON

- {} `"editor.snippetSuggestions": "top"` — list snippets first in IntelliSense
- {} `"emmet.includeLanguages": { "javascript": "javascriptreact" }` — Emmet in JSX
- {} `"editor.tabCompletion": "onlySnippets"` — Tab expands a prefix without the menu

IN OTHER EDITORS

- **JetBrains** — Live Templates ⌘J / Ctrl+J, Surround With ⌥⌘J; Emmet built in for HTML
- **Vim / Neovim** — LuaSnip or UltiSnips with a completion plugin (nvim-cmp, blink.cmp); emmet-vim for HTML

SNIPPET SYNTAX

SYNTAX	MEANS
`$1, $2`	Tab stops in order
`$0`	Final cursor
`${1:text}`	Placeholder text
`${1|a,b|}`	Choice
`$1 … $1`	Mirrored stop
`${TM_FILENAME_BASE}`	File name, no extension

EMMET ABBREVIATIONS

TYPE	EXPANDS TO
`!`	HTML5 boilerplate
`div.card`	
`ul>li*3`	ul with 3 li
`a[href=#]`	<a href="#">
`p{Hi}`	Hi
`m10`	margin: 10px; (CSS)

⚠ COMMON MISTAKES

- Editing a built-in snippet file inside an extension folder; updates overwrite it
- A `prefix` that matches a common word, so the snippet pops up while typing prose
- Unescaped `$` in a body (shell or PHP code) being read as a tab stop

✓ TIPS & BEST PRACTICES

- Commit team snippets as `.vscode/project.code-snippets` with a `scope` field
- Wrap selected text: use `${TM_SELECTED_TEXT}` and run **Insert Snippet** with a selection
- Use Emmet's **Wrap with Abbreviation** palette command to wrap existing HTML

QUICK SUMMARY

- `prefix` + `body` — a snippet
- `$1` / `$0` — tab stops / final cursor
- `*.code-snippets` — project snippets
- Emmet — `ul>li*3` + Tab

Sheet 11

Split Editors, Folding & View Options

split right ⌘\ · editor groups · preview vs pinned tabs · fold / unfold regions · sticky scroll · word wrap ⌥Z · minimap

ARRANGE THE EDITOR AREA

Split the editor into **editor groups** side by side or stacked; each group has its own tabs. Single-clicking a file opens a **preview** tab (italic) that the next click replaces; double-click or edit to keep it. **Folding** collapses blocks by indentation or language regions. **Sticky Scroll** pins the enclosing class and function names at the top.

NOTES

- Drag a tab to an editor edge to split there
- ⌘1 / Ctrl+1, ⌘2 / Ctrl+2… focus group 1, 2…
- `// #region` / `// #endregion` make foldable regions in most languages
- **Pin** a tab to keep it at the left and safe from Close All

LAYOUT MOVES

```
⌘\          / Ctrl+\           # split editor right
⌘K ⌘\       / Ctrl+K Ctrl+\    # split (choose direction)
⌘1 ⌘2       / Ctrl+1 Ctrl+2    # focus group 1 / 2
^⌘→         / Ctrl+Alt+→       # move tab to next group
⌘W          / Ctrl+W           # close tab
⇧⌘T         / Ctrl+Shift+T     # reopen closed tab
⌘K ↩        / Ctrl+K Enter     # keep preview tab open
⌥⌘[ / ⌥⌘]   / Ctrl+Shift+[ / ]  # fold / unfold block
⌘K ⌘0       / Ctrl+K Ctrl+0    # fold all
⌘K ⌘J       / Ctrl+K Ctrl+J    # unfold all
⌘K ⌘2       / Ctrl+K Ctrl+2    # fold level 2
⌥Z          / Alt+Z            # toggle word wrap
```

CODE EXPLAINED

→ Opening the same file in two groups gives two views of one buffer; edits show in both

→ `⌘K ⌘<n>` folds everything at level n, a quick outline of a long file

→ Word wrap is per window by default; set it per language in settings for Markdown

→ Reopen Closed Tab works across a session, not just the last tab

SETTINGS.JSON

{} `"workbench.editor.enablePreview": false` — every open is a real tab

{} `"editor.stickyScroll.enabled": true` — pin scope headers while scrolling

{} `"editor.minimap.enabled": false` — hide the code minimap

{} `"workbench.editor.limit.enabled": true` + `"workbench.editor.limit.value": 8` — cap open tabs

IN OTHER EDITORS

- **JetBrains** — Split Right from the tab menu, ⌘W / Ctrl+F4 close, fold ⌘- / Ctrl+-, unfold all ⇧⌘+ / Ctrl+Shift++
- **Vim / Neovim** — `:vsplit` / `:split`, `Ctrl+W h/j/k/l` to move, `zc` / `zo` fold, `zM` / `zR` all, `:set wrap`

TABS & GROUPS

ACTION	MACOS	WINDOWS / LINUX
Next / previous tab	^Tab / ^⇧Tab	Ctrl+Tab / +Shift
Close all in group	⌘K W	Ctrl+K W
Pin tab	⌘K ⇧↩	Ctrl+K Shift+Enter
Maximize group	double-click tab	double-click tab
Toggle vertical / horizontal	⌥⌘O	Shift+Alt+O

VIEW OPTIONS

OPTION	SETTING
Word wrap	editor.wordWrap
Minimap	editor.minimap.enabled
Sticky Scroll	editor.stickyScroll.enabled
Whitespace	editor.renderWhitespace
Indent guides	editor.guides.indentation
Bracket pairs	editor.guides.bracketPairs

⚠ COMMON MISTAKES

- Losing a file because it opened as a preview tab and the next click replaced it
- Splitting into four groups on a laptop screen and reading code at 40 characters wide
- Folding code to hide it in a review, then missing changes inside the folds

✓ TIPS & BEST PRACTICES

- Double-click a file in Explorer to open it as a kept tab straight away
- Use ⌘K ⌘2 / Ctrl+K Ctrl+2 to get an outline view of a long file
- Pin the two or three files you return to all day

QUICK SUMMARY

- `⌘\ / Ctrl+\` — split
- preview tab — italic, replaced on next open
- `⌘K ⌘0` / `⌘K ⌘J` — fold / unfold all
- `⌥Z / Alt+Z` — word wrap

Sheet 12

IntelliSense & Language Servers

trigger suggest ^Space · parameter hints · hover · inlay hints · language servers (LSP) · Problems panel ⇧⌘M

COMPLETIONS FROM A LANGUAGE SERVER

IntelliSense is VS Code's name for completions, parameter hints, hover info and diagnostics. Most of it comes from a **language server** that an extension starts (Pylance for Python, the TypeScript server, the Java Language Server). They speak the **Language Server Protocol (LSP)**, which is why one editor can support so many languages. Errors and warnings land in the **Problems** panel.

NOTES

- TypeScript / JavaScript, HTML, CSS and JSON support is built in
- Other languages: install the language's extension pack (Python, Java, C#, Go, Rust)
- **Inlay hints** show parameter names and inferred types in the code
- Language server logs: Output panel → pick the server in the dropdown

EXTENSION PICK

- **Error Lens** (Alexander) — prints each error and warning at the end of its line, so you don't need the Problems panel
- Language extension packs from Microsoft and the language teams (see each language set)

ASKING FOR HELP WHILE TYPING

```
^Space      / Ctrl+Space          # trigger suggestions
⇧⌘Space     / Ctrl+Shift+Space    # parameter hints
⌘K ⌘I       / Ctrl+K Ctrl+I       # show hover
Tab / ↩     / Tab / Enter         # accept suggestion
^Space again                      # expand suggestion details
⇧⌘M         / Ctrl+Shift+M        # Problems panel
F8 / ⇧F8    / F8 / Shift+F8       # next / previous problem

# Restart a stuck server (palette):
>TypeScript: Restart TS Server
>Python: Restart Language Server
>Developer: Reload Window
```

CODE EXPLAINED

- → Suggestions appear as you type; ^Space / Ctrl+Space forces them after you dismissed them
- → Parameter hints show the signature and bold the argument you're on
- → When completions stop or go wrong, restarting the server fixes most cases
- → **Reload Window** restarts all extensions without closing VS Code

SETTINGS.JSON

- {} `"editor.inlayHints.enabled": "onUnlessPressed"` — hide hints while holding ^⌥ / Ctrl+Alt
- {} `"editor.quickSuggestions": { "strings": "on" }` — suggest inside strings too
- {} `"editor.acceptSuggestionOnEnter": "off"` — only Tab accepts
- {} `"editor.suggest.preview": true` — show the completion inline before accepting

IN OTHER EDITORS

- **JetBrains** — completion ^Space / Ctrl+Space, parameter info ⌘P / Ctrl+P, quick doc F1 / Ctrl+Q; analysis is built into each IDE, not LSP
- **Vim / Neovim** — built-in LSP (`vim.lsp.enable()` in 0.11), `K` hover, `Ctrl+S` signature help in insert mode, nvim-lspconfig, blink.cmp / nvim-cmp

COMMON LANGUAGE SERVERS

LANGUAGE	EXTENSION / SERVER
TypeScript / JS	Built in (tsserver)
Python	Pylance
Java	Language Support for Java (Red Hat)
C#	C# Dev Kit
Go	Go (gopls)
Rust	rust-analyzer
C / C++	C/C++ or clangd

SUGGESTION ICONS

ICON	KIND
ƒ / ◇	Function / method
[x]	Variable
▭	Snippet
abc	Word from the file
◧	Class / interface
🔧	Property / field

⚠ COMMON MISTAKES

- Installing two servers for the same language (e.g. two Python servers) that fight over diagnostics
- Blaming VS Code for missing completions when the project's interpreter / SDK isn't selected
- Ignoring the Problems count in the Status Bar until CI fails

✓ TIPS & BEST PRACTICES

- Select the right interpreter or SDK first (Status Bar language item)
- Turn on inlay hints for parameter names when reading unfamiliar APIs
- Check the server's Output channel before reinstalling anything

QUICK SUMMARY

- `^Space / Ctrl+Space` — suggest
- `⇧⌘Space` — parameter hints
- LSP — one protocol, many languages
- Restart TS / language server — fixes stuck IntelliSense

Sheet 13

Refactoring & Code Actions

Rename Symbol F2 · Quick Fix ⌘. · extract function / variable · organize imports · source actions · refactor preview

LET THE LANGUAGE SERVER CHANGE CODE

Rename Symbol (F2) renames a variable, function or class everywhere it's used, across files, using the language server rather than text matching. **Code Actions** (the 💡 light bulb, ⌘. / Ctrl+.) offer **quick fixes** for errors (add import, fix typo) and **refactorings** for selections (extract function, extract constant, inline). **Source actions** act on the whole file: organize imports, fix all.

NOTES

- What's offered depends on the language server
- Rename shows a preview with ⇧↩ / Shift+Enter before applying
- Renaming a file in Explorer can update imports (TypeScript and others ask first)
- Refactor menu: ^⇧R / Ctrl+Shift+R

EXTRACT AND RENAME

```
# 1. select an expression
const total = items.reduce((s, i) => s + i.price * i.qty, 0);
#    ^⇧R / Ctrl+Shift+R → "Extract to function in module scope"
# 2. VS Code creates the function and puts you in rename
function calcTotal(items) { return items.reduce(...); }
const total = calcTotal(items);
# 3. F2 on calcTotal → type orderTotal → ⇧↩ to preview

⌘.      / Ctrl+.          # quick fix / code actions
F2      / F2              # rename symbol
^⇧R     / Ctrl+Shift+R    # refactor menu
⇧⌥O     / Shift+Alt+O     # organize imports
```

CODE EXPLAINED

→ Extract works on a selection: select first, then open the refactor menu

→ After extracting, the new name is selected for an immediate rename

→ Rename preview lists every file and line; untick any you don't want changed

→ Organize Imports sorts and removes unused imports for languages that support it

SETTINGS.JSON

{} `"editor.codeActionsOnSave": { "source.organizeImports": "explicit" }` — on manual save

{} `"editor.codeActionsOnSave": { "source.fixAll": "explicit" }` — run every auto-fix

{} `"javascript.updateImportsOnFileMove.enabled": "always"`

{} `"editor.lightbulb.enabled": "onCode"`

IN OTHER EDITORS

- **JetBrains** — Rename ⇧F6 / Shift+F6, Show Context Actions ⌥↩ / Alt+Enter, Refactor This ^T / Ctrl+Alt+Shift+T, Extract Method ⌥⌘M / Ctrl+Alt+M
- **Vim / Neovim** — Neovim 0.11 LSP defaults: `grn` rename, `gra` code action; or `vim.lsp.buf.rename()` / `.code_action()`

TYPICAL CODE ACTIONS

ACTION	WHERE IT APPEARS
`Add missing import`	On an unknown name
`Fix spelling`	Unknown identifier
`Extract function / method`	Selected expression or lines
`Extract constant / variable`	Selected expression
`Inline variable`	On a variable
`Convert to arrow / template string`	JS / TS
`Implement interface`	Class declaration

SOURCE ACTIONS

ACTION	KIND (CODEACTIONSONSAVE)
`Organize imports`	source.organizeImports
`Fix all`	source.fixAll
`ESLint fixes`	source.fixAll.eslint
`Ruff fixes`	source.fixAll.ruff
`Add missing imports`	source.addMissingImports

⚠ COMMON MISTAKES

- Renaming with Find & Replace and changing unrelated names in strings and comments
- Skipping the rename preview and renaming generated or vendored files too
- Using `"always"` for organize imports on save and getting noisy diffs in files you only looked at

✓ TIPS & BEST PRACTICES

- Use F2 even for local variables; it never touches a different variable with the same name
- Rename files from Explorer so imports update, not from the terminal
- Put source actions in `codeActionsOnSave` with `"explicit"` so autosave doesn't trigger them

QUICK SUMMARY

- `F2` — rename everywhere
- `⌘. / Ctrl+.` — quick fixes
- `^⇧R / Ctrl+Shift+R` — refactor
- `codeActionsOnSave` — fix and sort on save

Formatting & Linting

CONSISTENT CODE WITHOUT THINKING ABOUT IT

A **formatter** rewrites layout (indentation, quotes, line breaks); a **linter** reports problems and can auto-fix some. VS Code runs formatters through **Format Document** (⇧⌥F / Shift+Alt+F) or on save, and linters through extensions that feed the Problems panel. When several formatters handle a language, **editor.defaultFormatter** picks one.

NOTES

- Format Selection: ⌘K ⌘F / Ctrl+K Ctrl+F
- Format on save needs a formatter for that language, or nothing happens
- **Format Document With…** picks a formatter once and can set the default
- Tool configs (`.prettierrc` , `pyproject.toml`) win over editor settings

EXTENSION PICK

- **Prettier** (Prettier) — formatter for JS, TS, CSS, HTML, JSON, YAML, Markdown
- **ESLint** (Microsoft) — JS / TS linting and fixes · **Ruff** (Astral) — Python lint + format

.VSCODE/SETTINGS.JSON FOR A TEAM

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.codeActionsOnSave": {
      "source.fixAll.ruff": "explicit",
      "source.organizeImports.ruff": "explicit"
    }
  },
  "eslint.validate": ["javascript", "typescript", "typescriptreact"]
}
```

CODE EXPLAINED

- → The default formatter is an **extension ID** (`publisher.name`), shown on its Marketplace page
- → Format on save formats the file; code actions on save run the linter's fixes
- → Language blocks give Python its own formatter while everything else uses Prettier
- → Commit this file plus `.vscode/extensions.json` so everyone gets the same tools

SETTINGS.JSON

- {} `"editor.formatOnSaveMode": "modifications"` — format only changed lines (needs Git)
- {} `"editor.formatOnPaste": true`
- {} `"files.eol": "\n"` — LF line endings for new files

IN OTHER EDITORS

- **JetBrains** — Reformat Code ⌥⌘L / Ctrl+Alt+L, Settings → Tools → Actions on Save; Prettier and ESLint have built-in support
- **Vim / Neovim** — `gq` / `=` built in; conform.nvim for formatters, nvim-lint or LSP diagnostics for linters

FORMATTER / LINTER BY LANGUAGE

LANGUAGE	FORMAT	LINT
JS / TS	Prettier, Biome	ESLint, Biome
Python	Ruff, Black	Ruff
Java	Language server, google-java-format	Checkstyle, SonarLint
Go	gofmt (built into Go ext)	staticcheck
C#	C# Dev Kit / dotnet format	Roslyn analyzers
CSS	Prettier	Stylelint

.EDITORCONFIG

KEY	EXAMPLE
indent_style	space
indent_size	2
end_of_line	lf
insert_final_newline	true
[*.py]	section per file type

⚠ COMMON MISTAKES

- Two extensions both formatting on save and fighting over quotes or line length
- Formatter settings only in a teammate's User settings, so the repo flips style on every commit
- Turning on format on save in a legacy repo and reformatting whole files in unrelated PRs

✓ TIPS & BEST PRACTICES

- Set `formatOnSaveMode: "modifications"` when adopting a formatter on old code
- Keep the tool's own config file in the repo so CI formats the same way
- Check **Output → Prettier** (or the tool's channel) when formatting silently does nothing

QUICK SUMMARY

- `⇧⌥F / Shift+Alt+F` — format document
- `editor.defaultFormatter` — pick one per language
- `formatOnSave` + `codeActionsOnSave` — fix on save
- `.editorconfig` — basic style for every editor

Integrated Terminal

toggle ^` · new / split terminal · terminal profiles · shell integration · run selected text · terminal links · move to editor area

A REAL SHELL NEXT TO THE CODE

The **integrated terminal** runs your normal shell (zsh, bash, PowerShell) in the Panel, starting in the workspace folder. You can have many terminals, split them side by side, or move one into an editor tab. **Shell integration** marks each command, so you can jump between commands, see which failed, and rerun recent ones.

NOTES

- File paths and URLs in output are clickable (⌘ / Ctrl+Click)
- Run Selected Text sends the selection to the active terminal
- Terminals survive a window reload (persistent sessions)
- Pick a different shell from the + ▾ dropdown (terminal profiles)

TERMINAL SHORTCUTS

```
^`          / Ctrl+`           # toggle terminal
^⇧`         / Ctrl+Shift+`     # new terminal
⌘\           / Ctrl+Shift+5      # split terminal
⌘K           / (palette) Clear   # clear the terminal
⌘↑ / ⌘↓      / Ctrl+↑ / Ctrl+↓   # previous / next command
^R           / Ctrl+Alt+R        # run recent command
⇧⌘C          / Ctrl+Shift+C      # open external terminal

# Palette commands
>Terminal: Run Selected Text In Active Terminal
>Terminal: Move Terminal into Editor Area
>Terminal: Select Default Profile
```

CODE EXPLAINED

→ Command navigation (⌘↑ / Ctrl+↑) needs shell integration, which is on by default for common shells

→ The blue / red dot beside each command shows success or failure

→ Run Recent Command searches history from all sessions with shell integration

→ Moving a terminal into the editor area lets you split it next to code

SETTINGS.JSON

{} `"terminal.integrated.defaultProfile.osx": "zsh"` (`.windows` / `.linux` for others)

{} `"terminal.integrated.fontSize": 13`

{} `"terminal.integrated.cwd": "${workspaceFolder}"`

{} `"terminal.integrated.scrollback": 10000` — lines kept

IN OTHER EDITORS

- **JetBrains** — Terminal tool window ⌥F12 / Alt+F12; New Session, Split Right from its menu
- **Vim / Neovim** — `:terminal` (`:vsplit | terminal`), `Ctrl+\ Ctrl+N` to leave terminal mode; toggleterm.nvim plugin

DEFAULT SHELLS

OS	DEFAULT	OTHERS
macOS	zsh	bash, fish, pwsh
Windows	PowerShell	Command Prompt, Git Bash, WSL
Linux	$SHELL (bash)	zsh, fish, pwsh

FEATURES

FEATURE	USE
Terminal links	Click `src/app.ts:42` in output
Split	Server and watcher side by side
Rename / color	Right-click the terminal tab
Sticky scroll	Current command stays pinned
Run task	Tasks also run in terminals (sheet 18)

⚠ COMMON MISTAKES

- Running a long server in the only terminal, then killing it to run a command; open a second terminal
- Setting `PATH` in `.zshrc` and expecting already-open terminals to see it; open a new one
- Using the terminal's ⌘K / Ctrl+K and triggering a VS Code chord instead

✓ TIPS & BEST PRACTICES

- Name terminals ("server", "tests") so the dropdown makes sense
- Use Run Selected Text to try lines from a script without saving
- Turn on `terminal.integrated.stickyScroll.enabled` to keep the running command visible

QUICK SUMMARY

- `^` / Ctrl+`` — toggle terminal
- `^⇧`` — new terminal
- shell integration — jump and rerun commands
- profiles — choose the shell

Debugging Basics

PAUSE THE PROGRAM AND LOOK INSIDE

Set a **breakpoint** in the gutter, start debugging with **F5**, and the program pauses there. The **Run and Debug** view shows **Variables**, **Watch** expressions and the **Call Stack**; the **Debug Console** evaluates expressions in the paused frame. Each language's debugger comes from its extension (Python Debugger, Java, C#, Go; JavaScript is built in).

NOTES

- Without `launch.json`, F5 offers to debug the current file
- Hover a variable while paused to see its value
- **Logpoints** print a message without pausing or editing code
- JavaScript Debug Terminal debugs any Node command you run in it

EXTENSION PICK

- **Python Debugger** (Microsoft) — debugpy for Python
- **Debugger for Java** (Microsoft) · JavaScript debugging is built in

DEBUG KEYS

```
F9              / F9              # toggle breakpoint
F5              / F5              # start / continue
^F5             / Ctrl+F5         # run without debugging
F10             / F10             # step over
F11             / F11             # step into
⇧F11            / Shift+F11       # step out
⇧⌘F5            / Ctrl+Shift+F5   # restart
⇧F5             / Shift+F5        # stop

# Right-click the gutter:
Add Conditional Breakpoint…    # user.id == 42
Add Logpoint…                  # total={total}
Hit count                      # pause on the 10th hit
```

CODE EXPLAINED

- → Step over runs the line; step into enters a function call on it; step out finishes the current function
- → Conditional breakpoints use the language's own expression syntax
- → Logpoint messages put expressions in `{ }` and print to the Debug Console
- → Run without debugging still uses `launch.json` but skips breakpoints

SETTINGS.JSON

- {} `"debug.inlineValues": "on"` — show variable values beside the code while paused
- {} `"debug.toolBarLocation": "docked"` — put the debug toolbar in the Run and Debug view
- {} `"debug.openDebug": "openOnDebugBreak"` — show the view when a breakpoint hits

IN OTHER EDITORS

- **JetBrains** — Debug ^D / Shift+F9, breakpoint ⌘F8 / Ctrl+F8, step over F8, into F7, out ⇧F8 / Shift+F8, Evaluate Expression ⌥F8 / Alt+F8
- **Vim / Neovim** — nvim-dap + nvim-dap-ui (debug adapters, the same protocol VS Code uses); `:Termdebug` for gdb

RUN AND DEBUG VIEW

SECTION	SHOWS
`Variables`	Locals, globals, closures in the selected frame
`Watch`	Expressions you add, re-evaluated each pause
`Call Stack`	Frames; click one to inspect it
`Breakpoints`	All breakpoints; toggle, uncaught exceptions
`Debug Console`	REPL in the paused frame

BREAKPOINT TYPES

TYPE	PAUSES WHEN
`Line`	That line runs
`Conditional`	Expression is true
`Hit count`	Nth time
`Logpoint`	Never; logs instead
`Exception`	Raised / uncaught exception
`Function`	A named function is called

⚠ COMMON MISTAKES

- Debugging with `print` and restarting the program each time when one breakpoint would answer the question
- Stepping into library code by accident; use step out (⇧F11 / Shift+F11) or `justMyCode`
- Leaving uncaught-exception breakpoints off and missing where an error really started

✓ TIPS & BEST PRACTICES

- Use a logpoint in code you can't edit or restart easily
- Right-click a Call Stack frame → **Restart Frame** (where the language supports it) to rerun a function
- Pin values you check every pause in **Watch**

QUICK SUMMARY

- `F9` — breakpoint
- `F5` — start / continue
- `F10` / `F11` / `⇧F11` — over / into / out
- logpoint — print without pausing

launch.json Configurations

SAVE HOW TO START THE PROGRAM

`.vscode/launch.json` stores **debug configurations**: which debugger (`type`), whether to start the program (`launch`) or connect to one that's running (`attach`), and its arguments, environment and working folder. Pick a configuration in Run and Debug and press F5. **Compounds** start several at once, such as an API and a front end.

NOTES

- Create it: Run and Debug → **create a launch.json file**
- IntelliSense in the file lists every field for that `type`
- **Add Configuration…** inserts templates (Node, Python, Chrome…)
- Commit it so the team shares debug setups

.VSCODE/LAUNCH.JSON

```jsonc
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "API (Node)",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/src/server.js",
      "args": ["--port", "3000"],
      "envFile": "${workspaceFolder}/.env",
      "preLaunchTask": "npm: build",
      "skipFiles": ["<node_internals>/**"]
    },
    {
      "name": "Python: current file",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "justMyCode": true
    },
    { "name": "Attach :9229", "type": "node",
      "request": "attach", "port": 9229 }
  ],
  "compounds": [
    { "name": "Full stack", "configurations": ["API (Node)", "Web"] }
  ]
}
```

CODE EXPLAINED

- → `type` comes from the debugger extension: `node`, `debugpy`, `java`, `coreclr`, `go`, `chrome`
- → `${workspaceFolder}` and `${file}` are **variables** filled in at launch
- → `preLaunchTask` runs a task from `tasks.json` first (build, compile)
- → `attach` connects to a process already listening, e.g. `node --inspect`

SETTINGS.JSON

- {} `"debug.javascript.autoAttachFilter": "smart"` — auto-attach to Node started in the terminal
- {} `"launch": { … }` in User settings — configurations for every workspace
- {} `"debug.console.closeOnEnd": true`

IN OTHER EDITORS

- **JetBrains** — Run/Debug Configurations dialog, saved in `.idea/` or as shareable `.run/*.run.xml` files
- **Vim / Neovim** — nvim-dap `dap.configurations.<lang>`; it can also read `.vscode/launch.json` (`require("dap.ext.vscode").load_launchjs()`)

COMMON VARIABLES

VARIABLE	VALUE
`${workspaceFolder}`	Root of the open folder
`${file}`	Current file's path
`${fileDirname}`	Current file's folder
`${env:HOME}`	Environment variable
`${input:id}`	Prompt defined in `inputs`
`${command:pickProcess}`	Pick a process to attach

COMMON FIELDS

FIELD	SETS
`cwd`	Working directory
`args`	Command-line arguments
`env / envFile`	Environment
`console`	integratedTerminal / internalConsole
`stopOnEntry`	Pause on the first line
`postDebugTask`	Task after the session

⚠ COMMON MISTAKES

- Hard-coding absolute paths instead of `${workspaceFolder}`, so the config breaks on other machines
- Secrets in `env` inside a committed `launch.json`; use `envFile` and gitignore `.env`
- Using `launch` for a process started by Docker or a script; that's an `attach` job

✓ TIPS & BEST PRACTICES

- Start from **Add Configuration…** templates rather than an old blog post
- Use `${input:…}` with `promptString` / `pickString` for values that change each run
- Add a compound for the whole stack and debug front end and back end in one session

QUICK SUMMARY

- `launch` / `attach` — start or connect
- `${workspaceFolder}` — portable paths
- `preLaunchTask` — build first
- compounds — several at once

Tasks

NAMED COMMANDS YOU RUN FROM THE EDITOR

A **task** is a command (build, test, lint, start a watcher) with a name, saved in `.vscode/tasks.json`. VS Code also **auto-detects** tasks from npm scripts, Gulp, Make and others. Tasks run in a terminal; a **problem matcher** turns their output into clickable entries in Problems. Mark one task as the default **build** (⇧⌘B / Ctrl+Shift+B) or **test** task.

NOTES

- Run any task: palette **Tasks: Run Task** or the `task` prefix in Quick Open
- npm scripts appear as `npm: <script>` with no config
- `isBackground` + a matcher lets watch tasks signal when they're ready
- **Configure Task** turns a detected task into `tasks.json` you can edit

.VSCODE/TASKS.JSON

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "build",
      "type": "shell",
      "command": "npm run build",
      "group": { "kind": "build", "isDefault": true },
      "problemMatcher": ["$tsc"]
    },
    {
      "label": "test",
      "type": "shell",
      "command": "npx vitest run",
      "group": "test",
      "presentation": { "reveal": "always", "panel": "dedicated" }
    },
    {
      "label": "lint + test",
      "dependsOn": ["lint", "test"],
      "dependsOrder": "sequence"
    }
  ]
}
```

CODE EXPLAINED

→ `"shell"` runs through your shell (pipes, `&&`); `"process"` runs the program directly

→ `group.kind: "build"` + `isDefault` is what ⇧⌘B / Ctrl+Shift+B runs

→ `$tsc`, `$eslint-stylish`, `$gcc` are built-in problem matchers

→ `dependsOn` runs other tasks first, in parallel unless `dependsOrder: "sequence"`

SETTINGS.JSON

{} `"task.autoDetect": "on"` — find npm / Gulp / Make tasks automatically

{} `"task.problemMatchers.neverPrompt": true` — stop asking for a matcher

{} `"npm.scriptExplorerAction": "run"` — click an npm script to run it

IN OTHER EDITORS

- **JetBrains** — Run configurations (npm, Gradle, Maven, Shell Script) and External Tools; Before Launch steps chain them
- **Vim / Neovim** — `:make` with `makeprg` and `errorformat` (Vim's problem matcher), results in the quickfix list; overseer.nvim

TASK COMMANDS

COMMAND	MACOS	WINDOWS / LINUX
Run Build Task	⇧⌘B	Ctrl+Shift+B
Run Task	palette	palette
Run Test Task	palette	palette
Rerun Last Task	palette	palette
Terminate Task	palette	palette

USEFUL PROPERTIES

PROPERTY	USE
options.cwd	Run from another folder
options.env	Extra environment
runOptions.runOn	"folderOpen" to start on open
presentation.reveal	always / silent / never
isBackground	Long-running watcher
detail	Description in the picker

⚠ COMMON MISTAKES

- Copying a terminal command into a task without a problem matcher, so errors aren't clickable
- A watch task without `isBackground`, so a `preLaunchTask` waits forever
- Duplicating npm scripts as tasks instead of using the auto-detected `npm:` tasks

✓ TIPS & BEST PRACTICES

- Bind **Tasks: Rerun Last Task** to a key for a quick edit-run loop
- Use `runOn: "folderOpen"` for a watcher that should always be running
- Reference tasks from `launch.json` with `preLaunchTask` so debugging builds first

QUICK SUMMARY

- `tasks.json` — named commands
- `⇧⌘B / Ctrl+Shift+B` — default build task
- problem matcher — output → Problems
- `dependsOn` — chain tasks

The Testing View

Test Explorer · run / debug from the gutter · test output peek · rerun failed · coverage view · continuous run

ONE UI FOR EVERY TEST FRAMEWORK

The **Testing** view (beaker icon) lists tests that a framework extension discovers: pytest and unittest (Python), Jest and Vitest, JUnit, Playwright, .NET and more. Run or debug a single test from the ▷ in the gutter, a file, or everything. Failures open a **peek** with the assertion message and diff right at the failing line. Supported frameworks can also show **coverage** and run continuously on save.

NOTES

- Each framework needs its extension and a little config (e.g. enable pytest)
- Right-click the gutter ▷ for **Debug Test**
- Filter the tree by `@failed`, `@doc` (current file) or text
- pytest, Jest & Vitest, JUnit and Playwright sets cover each framework's setup

EXTENSION PICK

- **Python** (Microsoft) · **Vitest** (Vitest) · **Test Runner for Java** (Microsoft) · **Playwright Test** (Microsoft)
- Each plugs its tests into this same view

TESTING KEYS & SETUP

```
⌘; C     / Ctrl+; C        # run test at cursor
⌘; F     / Ctrl+; F        # run tests in current file
⌘; A     / Ctrl+; A        # run all tests
⌘; E     / Ctrl+; E        # rerun failed tests
⌘; L     / Ctrl+; L        # rerun last run
⌘; ⌘C    / Ctrl+; Ctrl+C   # debug test at cursor

# Python: enable pytest (palette)
>Python: Configure Tests → pytest → tests/
# writes to .vscode/settings.json:
"python.testing.pytestEnabled": true,
"python.testing.pytestArgs": ["tests"]
```

CODE EXPLAINED

- → The `⌘;` / `Ctrl+;` chord family covers run, debug and rerun
- → Configure Tests writes the framework choice to workspace settings
- → Coverage: use **Run with Coverage** (shield icon) where the extension supports it
- → Continuous run (eye icon) reruns affected tests when files change

SETTINGS.JSON

- {} `"testing.automaticallyOpenPeekView": "failureInVisibleDocument"`
- {} `"testing.defaultGutterClickAction": "debug"` — gutter click debugs instead of runs
- {} `"python.testing.pytestEnabled": true`

IN OTHER EDITORS

- **JetBrains** — gutter ▷ icons and Run ^ ⇧ R / Ctrl+Shift+F10 for the test at the cursor; Run tool window shows the tree
- **Vim / Neovim** — neotest (tree, gutter signs, output) or vim-test (`:TestNearest`, `:TestFile`)

FRAMEWORK → EXTENSION

FRAMEWORK	EXTENSION
`pytest / unittest`	Python
`Vitest`	Vitest
`Jest`	Jest (Orta)
`JUnit`	Test Runner for Java
`Playwright`	Playwright Test for VS Code
`xUnit / NUnit`	C# Dev Kit
`Go testing`	Go

TEST STATES

ICON	MEANS
▷	Not run yet
✔	Passed
✖	Failed (peek shows why)
⊘	Skipped
◌	Queued / running
⚠	Errored (setup or collection)

⚠ COMMON MISTAKES

- Seeing an empty Testing view and assuming no tests; usually the framework isn't enabled or discovery failed (check Output)
- Running the whole suite from the view when one test at the cursor would do
- Different test args in VS Code and CI, so tests pass in one and fail in the other

✓ TIPS & BEST PRACTICES

- Check **Output → Python Test Log** (or the framework's channel) when discovery fails
- Debug a failing test from the gutter instead of adding prints
- Keep framework config in its own file (`pyproject.toml`, `vitest.config.ts`) so CLI and editor agree

QUICK SUMMARY

- Testing view — tests from any framework
- `⌘; C / Ctrl+; C` — test at cursor
- `⌘; E / Ctrl+; E` — rerun failed
- peek — failure at the line

Git in VS Code

EVERYDAY GIT WITHOUT LEAVING THE EDITOR

The **Source Control** view (^⇧G / Ctrl+Shift+G) lists changed files. Click a file to see its diff; stage whole files with **+**, or single hunks and lines from the diff. Type a message and commit with ⌘↩ / Ctrl+Enter. The branch in the Status Bar switches or creates branches, and the **sync** button pulls then pushes. Colored bars in the gutter mark added, changed and deleted lines.

NOTES

- Uses the Git you have installed; it's the same repo as the command line
- **Source Control Graph** shows history and branches
- Click a gutter bar to see and revert that change inline
- The Git set covers what the commands do

EXTENSION PICK

- **GitLens** (GitKraken) — blame on every line, rich history, compare branches
- **Git Graph** (mhutchie) — a simple, free commit graph

WHERE TO FIND IT

ACTION	WHERE
`Diff of a file`	Source Control → click the file
`Stage / unstage file`	+ / - beside the file
`Switch / create branch`	Status Bar branch name
`Pull, push, fetch`	··· menu in Source Control
`Stash`	··· → Stash
`History of a file`	Explorer → Timeline view
`Blame (built in)`	Status Bar / hover (git.blame settings)

COMMIT PART OF A FILE

1. Open Source Control (^⇧G / Ctrl+Shift+G) and click the changed file
2. In the diff, select the lines you want in this commit
3. Right-click → **Stage Selected Ranges** (⌘K ⌥⌘S / Ctrl+K Ctrl+Alt+S)
4. Type the message; first line short, blank line, then detail
5. Commit with ⌘↩ / Ctrl+Enter (the ✓ button)
6. Click the sync icon in the Status Bar, or ··· → **Push**

SETTINGS.JSON

- {} `"git.autofetch": true` — fetch in the background so behind/ahead counts are current
- {} `"git.enableSmartCommit": true` — commit all changes when nothing is staged
- {} `"git.confirmSync": false` — skip the sync confirmation
- {} `"git.blame.editorDecoration.enabled": true` — inline blame on the current line

IN OTHER EDITORS

- **JetBrains** — Commit tool window ⌘K / Ctrl+K, partial commits by ticking lines, Git Log ⌘9 / Alt+9, branches popup in the status bar
- **Vim / Neovim** — vim-fugitive (`:Git` , `:Git blame` , stage hunks with `s`), gitsigns.nvim for gutter signs and hunk staging; lazygit in a terminal

FILE STATUS LETTERS

LETTER	MEANS
`M`	Modified
`U`	Untracked
`A`	Added (staged new file)
`D`	Deleted
`R`	Renamed
`C`	Conflict

GUTTER BARS

MARK	MEANS
`Green bar`	Added lines
`Blue bar`	Changed lines
`Red triangle`	Deleted lines here
`Click a bar`	Inline diff with revert / stage

⚠ COMMON MISTAKES

- Committing with smart commit on and sweeping in files you didn't mean to stage
- Discarding changes from the ↺ button without reading the prompt; it can't be undone
- Pushing with the sync button on a shared branch without pulling the latest first

✓ TIPS & BEST PRACTICES

- Stage by hunk or line to keep commits focused
- Review the full diff of what you staged before committing (click the file under **Staged Changes**)
- Turn on `git.autofetch` so you see when the remote moved

QUICK SUMMARY

- `^⇧G / Ctrl+Shift+G` — Source Control
- Stage Selected Ranges — commit part of a file
- `⌘↩ / Ctrl+Enter` — commit
- Status Bar branch — switch / create

Diff & Merge Editor

compare files · inline vs side-by-side diff · `code -d` · 3-way merge editor · accept incoming / current · resolve conflicts

SEE CHANGES AND SETTLE CONFLICTS

The **diff editor** shows two versions side by side (or inline), with moved code detected and unchanged regions collapsed. When Git reports a **merge conflict**, VS Code highlights the conflict blocks with **Accept Current / Accept Incoming / Accept Both** links, or opens the **3-way merge editor**: incoming and current on top, the result below.

NOTES

- Compare any two files: select both in Explorer → right-click → **Compare Selected**
- **Current** = your branch (HEAD); **Incoming** = what you're merging in
- The merge editor shows the conflict count and a **Complete Merge** button
- Compare with clipboard: palette **File: Compare Active File with Clipboard**

RESOLVE A MERGE CONFLICT

1. Run the merge or rebase; conflicted files show **C** in Source Control
2. Open a file and click **Resolve in Merge Editor**
3. For each conflict, tick **Accept Incoming**, **Accept Current** or both
4. Edit the **Result** pane by hand where neither side is right
5. Click **Complete Merge**; the file is staged
6. Run the tests, then commit (or **Continue** a rebase)

WHERE TO FIND IT

ACTION	WHERE
`Diff of a changed file`	Source Control → click it
`Inline vs side by side`	Diff toolbar ··· → Inline View
`Next / previous change`	⌥F5 / ⇧⌥F5 (Alt+F5 / Shift+Alt+F5)
`Compare two files`	Explorer → Compare Selected
`Diff from terminal`	`code -d a.txt b.txt`
`Merge editor`	**Resolve in Merge Editor** button

SETTINGS.JSON

- {} `"diffEditor.renderSideBySide": true` (false = inline)
- {} `"diffEditor.ignoreTrimWhitespace": false` — show whitespace changes
- {} `"git.mergeEditor": true` — open conflicts in the merge editor
- {} `"diffEditor.hideUnchangedRegions.enabled": true`

IN OTHER EDITORS

- **JetBrains** — Merge dialog with three panes (⇇ accept left, ⇉ accept right) from the Git conflicts popup; Compare with clipboard
- **Vim / Neovim** — `vimdiff` / `nvim -d`, `:diffget LO` / `RE` in a 3-way merge, `]c` / `[c` next change; diffview.nvim plugin

CONFLICT BLOCK

MARKER	SECTION
`<<<<<<< HEAD`	Current (yours)
`=======`	Divider
`>>>>>>> branch`	Incoming (theirs)
`Accept Both`	Keeps current then incoming
`Compare Changes`	Diff of the two sides

CURRENT VS INCOMING

OPERATION	CURRENT	INCOMING
`git merge feature`	your branch	feature
`git rebase main`	main (being replayed onto)	your commits
`git cherry-pick X`	your branch	commit X

⚠ COMMON MISTAKES

- Clicking **Accept Both** on two edits of the same line and keeping duplicate code
- Forgetting that during a rebase "current" and "incoming" swap meaning
- Committing a file that still contains `<<<<<<<` markers; the merge editor's count prevents this

✓ TIPS & BEST PRACTICES

- Use **Compare Changes** on a conflict to see what each side changed from the base
- Turn off whitespace-ignoring when a diff looks empty but Git says the file changed
- Use `code -d` as Git's difftool: `git config --global diff.tool vscode`

QUICK SUMMARY

- diff editor — side by side or inline
- Current = HEAD · Incoming = theirs
- merge editor — 3-way with a Result pane
- `code -d` — diff from the terminal

GitHub Integration

PULL REQUESTS AND ISSUES IN THE EDITOR

Signing in with GitHub (Accounts menu) lets VS Code **publish** a repo, clone private ones and push over HTTPS. The **GitHub Pull Requests** extension adds a view for PRs and issues: check out a PR as a local branch, review the diff with full IntelliSense, leave comments in the gutter and approve or request changes. Pressing **.** on any github.com repo opens **github.dev**, a browser VS Code for quick edits.

NOTES

- **Publish to GitHub** in Source Control creates the remote repo and pushes
- Clone: palette **Git: Clone → Clone from GitHub**
- github.dev has no terminal or debugger; **Codespaces** adds them
- The GitHub set covers PRs, Issues and Actions on the website

EXTENSION PICK

- **GitHub Pull Requests** (GitHub) — PR and issue views, review comments, check out PRs
- **GitHub Actions** (GitHub) — workflow runs and logs in the Side Bar

REVIEW A PULL REQUEST LOCALLY

1. Open the GitHub view and expand **Pull Requests → Waiting for my review**
2. Click the PR → **Checkout**; VS Code switches to its branch
3. Open **Files Changed**; each file opens as a diff you can navigate and run
4. Click **+** in the gutter to comment on a line, or select lines first
5. Use the PR description page to **Approve**, **Comment** or **Request Changes**
6. Switch back with the branch picker, or **Exit Review Mode**

WHERE TO FIND IT

ACTION	WHERE
Sign in	Accounts icon (bottom of Activity Bar)
Publish a repo	Source Control → Publish to GitHub
PRs & issues	GitHub view (after installing the extension)
Create a PR	GitHub view → **+**, or after pushing a branch
Open on github.com	Right-click a line → **Copy GitHub Permalink**
Browser editor	Press **.** on a repo page

SETTINGS.JSON

- `{} "githubPullRequests.defaultMergeMethod": "squash"`
- `{} "github.gitProtocol": "ssh"` — clone with SSH instead of HTTPS
- `{} "git.openRepositoryInParentFolders": "always"`

IN OTHER EDITORS

- **JetBrains** — Pull Requests tool window and GitHub sign-in built in (Settings → Version Control → GitHub); Create Pull Request from the Git menu
- **Vim / Neovim** — octo.nvim (PRs and issues in buffers), the `gh` CLI in a terminal, vim-rhubarb `:GBrowse` for permalinks

WEB EDITORS

OPTION	GIVES YOU
github.dev (.)	Editor + Git commits, free, no compute
Codespaces	Full VS Code + terminal on a cloud VM
vscode.dev	Browser editor for any repo or local folder

REVIEW COMMENT ACTIONS

ACTION	RESULT
Add Comment	Posted at once
Start Review	Pending until you submit
Suggest a Change	Suggestion block the author can apply
Resolve conversation	Collapses the thread

⚠ COMMON MISTAKES

- Reviewing only on the website and never running the PR's code
- Leaving review comments as **Start Review** and forgetting to submit, so the author never sees them
- Editing on github.dev and expecting to run tests there

✓ TIPS & BEST PRACTICES

- Check out PRs locally for anything bigger than a typo
- Use **Copy GitHub Permalink** to share a line in chat
- Create the PR from VS Code right after pushing; it fills the title from your commits

QUICK SUMMARY

- Publish to GitHub — create remote + push
- GitHub Pull Requests ext — review in the editor
- Checkout — run the PR locally
- **.** — github.dev

Managing Extensions

ADD ONLY WHAT YOU USE

Almost every language, linter, theme and tool comes as an **extension** from the Marketplace (VSCodium and some forks use **Open VSX**). Install from the **Extensions** view (⇧⌘X / Ctrl+Shift+X) or the CLI. Recommend extensions to a team with `.vscode/extensions.json`; VS Code prompts anyone who opens the repo. Disable extensions **per workspace** to keep other projects fast.

NOTES

- An extension ID is `publisher.name`, e.g. `ms-python.python`
- Check the publisher's verified ✓ badge and install count
- **Extension Bisect** finds the extension causing a problem by halving the set
- Extension settings: gear icon → **Settings**, or `@ext:` in the Settings search

EXTENSION PICK

- **EditorConfig** (EditorConfig) — applies `.editorconfig` rules
- **Code Spell Checker** (Street Side Software) — catches typos in code and comments

INSTALL AND RECOMMEND

```
code --install-extension ms-python.python
code --install-extension esbenp.prettier-vscode@11.0.0
code --uninstall-extension some.extension
code --list-extensions > extensions.txt

# .vscode/extensions.json (commit this)
{
  "recommendations": [
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "vitest.explorer"
  ],
  "unwantedRecommendations": ["hookyqr.beautify"]
}
```

CODE EXPLAINED

- → `@version` installs a specific version, useful when a new release breaks something
- → `--list-extensions` output can be piped back into `--install-extension` on a new machine
- → Recommendations show under **Recommended** in the Extensions view
- → `unwantedRecommendations` stops VS Code suggesting an extension for this repo

SETTINGS.JSON

- {} `"extensions.autoUpdate": "onlyEnabledExtensions"`
- {} `"extensions.ignoreRecommendations": false`
- {} `"extensions.autoCheckUpdates": true`

IN OTHER EDITORS

- **JetBrains** — Plugins ⌘, / Ctrl+Alt+S → Plugins (JetBrains Marketplace); required plugins in `.idea/externalDependencies.xml`
- **Vim / Neovim** — plugin managers: lazy.nvim, vim-plug, packer; Mason for language servers and formatters

EXTENSIONS VIEW FILTERS

TYPE	SHOWS
`@installed`	Installed
`@enabled / @disabled`	By state
`@recommended`	Workspace + general picks
`@updates`	Waiting for updates
`@category:themes`	By category
`@sort:installs`	Most installed

WHEN VS CODE MISBEHAVES

TRY	HOW
`Without extensions`	`code --disable-extensions`
`Find the culprit`	Help → Start Extension Bisect
`See what's slow`	Developer: Show Running Extensions
`Reload`	Developer: Reload Window

⚠ COMMON MISTAKES

- Installing from unknown publishers; extensions run with your user's permissions
- Two extensions doing the same job (two Python formatters, two Git UIs) and slowing startup
- Recommending extensions only in a README instead of `extensions.json`

✓ TIPS & BEST PRACTICES

- Disable heavy extensions per workspace instead of uninstalling them
- Use **Show Running Extensions** to spot slow activation times
- Keep `extensions.json` short: only what the project actually needs

QUICK SUMMARY

- `⇧⌘X / Ctrl+Shift+X` — Extensions view
- `publisher.name` — extension ID
- `extensions.json` — team recommendations
- Extension Bisect — find a bad extension

Workspaces & Multi-Root

ONE WINDOW, ONE OR MANY FOLDERS

Opening a folder makes it the **workspace**: its `.vscode/` folder holds settings, launch and task configs. A **multi-root workspace** opens several folders in one window (say, `api/` and `web/` from different repos), saved as a `.code-workspace` file with its own settings. **Workspace Trust** asks before running code from a folder you just downloaded; in Restricted Mode, tasks, debugging and some extensions are off.

NOTES

- **File → Add Folder to Workspace** turns a window into multi-root
- **Save Workspace As…** writes the `.code-workspace` file
- Each root folder can keep its own `.vscode/settings.json`
- Trust a parent folder (e.g. `~/projects`) to trust everything under it

TEAM.CODE-WORKSPACE

```
{
  "folders": [
    { "path": "api" },
    { "path": "web" },
    { "path": "../shared-lib", "name": "shared" }
  ],
  "settings": {
    "files.exclude": { "**/node_modules": true },
    "editor.formatOnSave": true
  },
  "extensions": {
    "recommendations": ["dbaeumer.vscode-eslint"]
  },
  "launch": {
    "compounds": [
      { "name": "API + Web",
        "configurations": ["API", "Web"] }
    ]
  }
}
```

CODE EXPLAINED

→ Paths are relative to the `.code-workspace` file; `name` changes the label in Explorer

→ Workspace-file settings apply to all roots; folder settings can override them

→ `launch` and `tasks` can live in the workspace file to span folders

→ Open it with `code team.code-workspace` or double-click

SETTINGS.JSON

{} `"security.workspace.trust.untrustedFiles": "prompt"`

{} `"files.exclude": { "**/.DS_Store": true }` — hide from Explorer

{} `"explorer.fileNesting.enabled": true` — group `app.ts` with `app.test.ts`

IN OTHER EDITORS

- **JetBrains** — a project = a folder with `.idea/`; attach more projects to one window (PyCharm, WebStorm) or use modules (IntelliJ)
- **Vim / Neovim** — `:cd` / `:tcd` per tab, `:mksession` to save layout; a "workspace" is just the current directory

THE .VSCODE/ FOLDER

FILE	HOLDS	COMMIT?
`settings.json`	Workspace settings	Yes (shared ones)
`launch.json`	Debug configurations	Yes
`tasks.json`	Tasks	Yes
`extensions.json`	Recommendations	Yes
`mcp.json`	MCP servers	Yes, no secrets
`*.code-snippets`	Project snippets	Yes

RESTRICTED MODE BLOCKS

FEATURE	RESTRICTED
`Tasks`	Off
`Debugging`	Off
`Workspace settings that run code`	Ignored
`Extensions without trust support`	Disabled
`Reading / editing files`	Allowed

⚠ COMMON MISTAKES

- Trusting every folder automatically, including ones from email attachments or random repos
- Committing `.code-workspace` files with paths that only exist on your machine
- Putting secrets in `.vscode/settings.json` or `mcp.json` and committing them

✓ TIPS & BEST PRACTICES

- Use a multi-root workspace for a front end and back end in separate repos
- Turn on file nesting to tidy Explorer in projects with many generated files
- Gitignore `.vscode/*` but un-ignore the shared files: `!.vscode/settings.json`, `!.vscode/extensions.json`

QUICK SUMMARY

- folder — one root with `.vscode/`
- `.code-workspace` — many roots + shared settings
- Workspace Trust — Restricted Mode for unknown code
- `.vscode/` — settings, launch, tasks, extensions, mcp

Remote Development

THE UI STAYS LOCAL, THE CODE RUNS ELSEWHERE

With the **Remote Development** extensions, VS Code runs a small server where the code lives and keeps the UI on your machine. **Remote - SSH** opens folders on a server, **WSL** opens folders inside Linux on Windows, and **Dev Containers** opens a folder inside a Docker container defined by `devcontainer.json`. Terminals, debuggers and extensions run on the remote side, so you get the remote OS and tools.

NOTES

- The green >< button (bottom left) opens every remote option
- Extensions install "locally" (themes, keymaps) or "remotely" (language servers)
- **Codespaces** = a dev container on GitHub's cloud
- **Remote Tunnels** reach a machine with no SSH access through GitHub sign-in

EXTENSION PICK

- **Remote Development** (Microsoft) — pack with Remote - SSH, WSL and Dev Containers
- **Dev Containers** alone if you only need containers

.DEVCONTAINER/DEVCONTAINER.JSON

```jsonc
{
  "name": "Node + Postgres",
  "image": "mcr.microsoft.com/devcontainers/typescript-node:22",
  "features": {
    "ghcr.io/devcontainers/features/github-cli:1": {}
  },
  "forwardPorts": [3000, 5432],
  "postCreateCommand": "npm ci",
  "customizations": {
    "vscode": {
      "extensions": ["dbaeumer.vscode-eslint"],
      "settings": { "editor.formatOnSave": true }
    }
  },
  "remoteUser": "node"
}
```

CODE EXPLAINED

- → `image` (or `build.dockerfile`, or `dockerComposeFile`) defines the container
- → **Features** add tools (CLI, language runtimes) without writing a Dockerfile
- → `postCreateCommand` runs once after the container is built
- → Extensions listed under `customizations.vscode` install inside the container

SETTINGS.JSON

- {} `"remote.SSH.remotePlatform": { "build-box": "linux" }`
- {} `"remote.autoForwardPorts": true` — forward ports when a server starts
- {} `"dev.containers.dockerPath": "podman"` — use Podman

IN OTHER EDITORS

- **JetBrains** — JetBrains Gateway / Remote Development over SSH, Dev Containers support in IntelliJ-based IDEs, WSL interpreters
- **Vim / Neovim** — run Vim directly on the server over `ssh`; `nvim scp://host//path` for single files; distant.nvim

REMOTE OPTIONS

OPTION	RUNS ON	NEEDS
`Remote - SSH`	Any SSH host	SSH access
`WSL`	Linux on Windows	WSL installed
`Dev Containers`	Local Docker container	Docker / Podman
`Codespaces`	GitHub cloud VM	GitHub account
`Remote Tunnels`	Any machine	GitHub / Microsoft sign-in

PALETTE COMMANDS

COMMAND	DOES
`Remote-SSH: Connect to Host…`	Open a server folder
`WSL: Connect to WSL`	New Linux window
`Dev Containers: Reopen in Container`	Build and open
`Dev Containers: Rebuild Container`	Apply config changes
`Ports: Focus on Ports View`	See forwarded ports

⚠ COMMON MISTAKES

- Opening a Windows folder (`/mnt/c/…`) from WSL; file access is slow, keep code in the Linux file system
- Editing `devcontainer.json` and forgetting to **Rebuild Container**
- Installing a language extension locally and wondering why it does nothing in the remote window

✓ TIPS & BEST PRACTICES

- Commit `.devcontainer/` so new contributors get a working setup in one click
- Use `~/.ssh/config` host aliases; Remote - SSH reads them
- Use Features instead of custom Dockerfiles for common tools

QUICK SUMMARY

- Remote - SSH — folders on a server
- WSL — Linux on Windows
- Dev Containers — `devcontainer.json`
- Codespaces — a dev container in the cloud

Sheet 26

Copilot Completions & Chat

inline suggestions · next edit suggestions · Chat view ^⌘I · inline chat ⌘I · Ask / Agent / Plan · # context · `.github/copilot-instructions.md`

AI BUILT INTO THE EDITOR

VS Code ships with **GitHub Copilot** built in (sign in with GitHub; a free tier exists). **Inline suggestions** appear as ghost text while you type; **next edit suggestions** predict the next change you'll make elsewhere. The **Chat** view (^⌘I / Ctrl+Alt+I) answers questions and makes edits; **inline chat** (⌘I / Ctrl+I) works on the selection in place. Pick an agent in Chat: **Ask** (read-only answers), **Agent** (edits and runs tools) or **Plan** (writes a plan to review first).

NOTES

- Tab accepts a suggestion; ⌘→ / Ctrl+→ accepts one word
- Add context with `#file`, `#selection`, `#codebase`, or drag files into Chat
- `/` commands in chat: `/explain`, `/fix`, `/tests`, `/plan`
- Pick the model from the model picker in the Chat input

CUSTOM INSTRUCTIONS & PROMPT FILES

```
# .github/copilot-instructions.md  (always added)
- Use TypeScript strict mode and named exports.
- Tests use Vitest; put them next to the file as *.test.ts.
- Never edit files under src/generated/.

# .github/instructions/api.instructions.md
---
applyTo: "src/api/**"
---
Return problem+json errors (see docs/errors.md).

# .github/prompts/new-endpoint.prompt.md → /new-endpoint
---
description: Scaffold a REST endpoint with tests
---
Create a handler, route and Vitest test for ${input:name}.
```

CODE EXPLAINED

- → `copilot-instructions.md` is added to every chat request in the repo
- → `*.instructions.md` with `applyTo` globs add rules only for matching files
- → Prompt files become `/name` commands you can run in Chat
- → `AGENTS.md` at the repo root is read too, so other agents can share the same rules

SETTINGS.JSON

- {} `"github.copilot.enable": { "*": true, "markdown": false }` — per-language on/off
- {} `"github.copilot.nextEditSuggestions.enabled": true`
- {} `"editor.inlineSuggest.enabled": true`
- {} `"chat.agent.maxRequests": 25` — steps before Agent asks to continue

IN OTHER EDITORS

- **JetBrains** — JetBrains AI Assistant and the GitHub Copilot plugin; inline completion, chat tool window, project rules files
- **Vim / Neovim** — copilot.vim / copilot.lua for completions; CodeCompanion.nvim or avante.nvim for chat with any model

COPILOT KEYS

ACTION	MACOS	WINDOWS / LINUX
Accept suggestion	Tab	Tab
Accept next word	⌘→	Ctrl+→
Next / previous suggestion	⌥] / ⌥[	Alt+] / Alt+[
Dismiss	Esc	Esc
Open Chat	^⌘I	Ctrl+Alt+I
Inline chat	⌘I	Ctrl+I

BUILT-IN AGENTS

AGENT	DOES
Ask	Answers and researches; read-only tools
Agent	Edits files, runs commands and tools, checks results
Plan	Researches and writes a step-by-step plan for you to approve
Custom agents	`.github/agents/*.agent.md` with their own tools and instructions

⚠ COMMON MISTAKES

- Accepting a long suggestion without reading it; review AI code like a teammate's PR
- Asking Chat about "this code" without adding the file or selection as context
- Writing team rules in personal settings instead of `copilot-instructions.md` in the repo

✓ TIPS & BEST PRACTICES

- Start bigger changes in **Plan**, then hand the plan to **Agent**
- Keep `copilot-instructions.md` short and specific; long files get diluted
- Use inline chat (⌘I / Ctrl+I) for small, local edits

QUICK SUMMARY

- ghost text — Tab to accept
- `^⌘I / Ctrl+Alt+I` — Chat · `⌘I / Ctrl+I` — inline
- Ask · Agent · Plan — built-in agents
- `copilot-instructions.md` — repo rules

Agent Mode, MCP & Other AI Tools

AGENTS THAT USE TOOLS

In **Agent** mode the model can read and edit files, run terminal commands and call **tools**. Each tool call that can change things asks for **approval** unless you allow it. **MCP servers** (Model Context Protocol) add more tools, such as a database, a browser or an issue tracker, configured in `.vscode/mcp.json`. VS Code can also host other agents: the session picker offers harnesses such as **Copilot**, **Local**, **Claude** and **Codex**, and the **Claude Code** extension adds its own panel.

NOTES

- Tool approvals can be once, for the session, for the workspace, or always
- The tools icon in the Chat input turns individual tools on or off
- **MCP: List Servers** in the palette starts, stops and shows logs
- The Agentic Coding set covers prompting and workflows in depth

EXTENSION PICK

- **Claude Code for VS Code** (Anthropic) — Claude Code in a side panel with inline diffs
- **GitHub Copilot Chat** (GitHub) — Chat, agents and MCP support

.VSCODE/MCP.JSON

```
{
  "inputs": [
    { "id": "gh-token", "type": "promptString",
      "description": "GitHub token", "password": true }
  ],
  "servers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    },
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": { "Authorization": "Bearer ${input:gh-token}" }
    }
  }
}
```

CODE EXPLAINED

- → `servers` lists each MCP server: `command` + `args` for a local (stdio) one, `type: "http"` + `url` for a remote one
- → `inputs` prompt for secrets once and store them securely, so the file can be committed
- → Servers can also be installed from the Extensions view (`@mcp`) or added to User settings
- → Custom agents (`.github/agents/*.agent.md`) can limit which tools an agent may use

SETTINGS.JSON

- {} `"chat.agent.enabled": true`
- {} `"chat.mcp.discovery.enabled": true` — reuse MCP configs from other apps
- {} `"github.copilot.chat.codeGeneration.useInstructionFiles": true`

IN OTHER EDITORS

- **JetBrains** — Junie and AI Assistant agents, MCP client support in AI Assistant settings
- **Vim / Neovim** — CodeCompanion.nvim or avante.nvim with MCP via mcphub.nvim; or run Claude Code / Codex CLI in a terminal split

APPROVAL CHOICES

CHOICE	SCOPE
`Allow once`	This call only
`Allow in this session`	Until the chat ends
`Allow in this workspace`	Saved for the repo
`Always allow`	Every workspace
`Skip / deny`	Model must try another way

AI CONFIG FILES

FILE	PURPOSE
`.vscode/mcp.json`	MCP servers for the workspace
`.github/copilot-instructions.md`	Rules for every request
`.github/agents/*.agent.md`	Custom agents
`.github/prompts/*.prompt.md`	Reusable / prompts
`AGENTS.md / CLAUDE.md`	Rules other agents also read

⚠ COMMON MISTAKES

- Turning on "always allow" for terminal commands in a repo you don't fully trust
- Committing tokens in `mcp.json` instead of using `inputs` or environment variables
- Adding a dozen MCP servers so the model has too many tools to pick from well

✓ TIPS & BEST PRACTICES

- Enable only the MCP servers a task needs
- Review the diff Agent produces in Source Control before committing
- Use a custom agent with a read-only tool set for reviews and research

QUICK SUMMARY

- Agent — edits + tools with approvals
- `.vscode/mcp.json` — MCP servers
- `inputs` — secrets without committing them
- Claude Code / Codex — other agents in VS Code

Sheet 28

Quick Reference — Keyboard Shortcuts

macOS and Windows / Linux columns · general · editing · multi-cursor · navigation · search · debug · terminal

GENERAL (03, 02)

ACTION	MACOS	WIN / LINUX
Command Palette	⇧⌘P	Ctrl+Shift+P
Quick Open file	⌘P	Ctrl+P
Settings	⌘,	Ctrl+,
Keyboard Shortcuts	⌘K ⌘S	Ctrl+K Ctrl+S
Side Bar	⌘B	Ctrl+B
Panel	⌘J	Ctrl+J
Terminal	^`	Ctrl+`
Zen Mode	⌘K Z	Ctrl+K Z
Chat	^⌘I	Ctrl+Alt+I

EDITING (06, 07)

ACTION	MACOS	WIN / LINUX
Move line	⌥↑ / ⌥↓	Alt+↑ / ↓
Copy line	⇧⌥↑ / ⇧⌥↓	Shift+Alt+↑ / ↓
Delete line	⇧⌘K	Ctrl+Shift+K
Comment	⌘/	Ctrl+/
Next match	⌘D	Ctrl+D
All matches	⇧⌘L	Ctrl+Shift+L
Cursor above / below	⌥⌘↑ / ↓	Ctrl+Alt+↑ / ↓
Expand selection	^⇧⌘→	Shift+Alt+→
Format	⇧⌥F	Shift+Alt+F

NAVIGATION (08, 11)

ACTION	MACOS	WIN / LINUX
Definition	F12	F12
Peek	⌥F12	Alt+F12
References	⇧F12	Shift+F12
Symbol in file	⇧⌘O	Ctrl+Shift+O
Symbol anywhere	⌘T	Ctrl+T
Go to line	^G	Ctrl+G
Back / forward	^- / ^⇧-	Alt+← / →
Split editor	⌘\	Ctrl+\
Reopen tab	⇧⌘T	Ctrl+Shift+T

SEARCH & CODE (09, 12, 13)

ACTION	MACOS	WIN / LINUX
Find	⌘F	Ctrl+F
Replace	⌥⌘F	Ctrl+H
Search files	⇧⌘F	Ctrl+Shift+F
Replace in files	⇧⌘H	Ctrl+Shift+H
Suggest	^Space	Ctrl+Space
Quick fix	⌘.	Ctrl+.
Rename	F2	F2
Refactor	^⇧R	Ctrl+Shift+R
Next problem	F8	F8

DEBUG, TEST & TASKS (16–19)

ACTION	MACOS	WIN / LINUX
Breakpoint	F9	F9
Start / continue	F5	F5
Step over / into	F10 / F11	F10 / F11
Step out	⇧F11	Shift+F11
Stop	⇧F5	Shift+F5
Build task	⇧⌘B	Ctrl+Shift+B
Test at cursor	⌘; C	Ctrl+; C
Rerun failed	⌘; E	Ctrl+; E
Source Control	^⇧G	Ctrl+Shift+G

TERMINAL & TABS (11, 15)

ACTION	MACOS	WIN / LINUX
New terminal	^⇧`	Ctrl+Shift+`
Split terminal	⌘\	Ctrl+Shift+5
Previous command	⌘↑	Ctrl+↑
Run recent command	^R	Ctrl+Alt+R
Next / previous tab	^Tab	Ctrl+Tab
Close tab	⌘W	Ctrl+W
Focus group 1 / 2	⌘1 / ⌘2	Ctrl+1 / 2
Fold / unfold all	⌘K ⌘O / ⌘K ⌘J	Ctrl+K Ctrl+O / J
Word wrap	⌥Z	Alt+Z

GIT & DIFF (20, 21)

ACTION	MACOS	WIN / LINUX
Source Control	^⇧G	Ctrl+Shift+G
Commit	⌘↩ (in message)	Ctrl+Enter
Stage selected ranges	⌘K ⌥⌘S	Ctrl+K Ctrl+Alt+S
Next change in diff	⌥F5	Alt+F5
Previous change	⇧⌥F5	Shift+Alt+F5
Compare with clipboard	palette	palette

COPILOT & CHAT (26, 27)

ACTION	MACOS	WIN / LINUX
Accept suggestion	Tab	Tab
Accept word	⌘→	Ctrl+→
Next suggestion	⌥]	Alt+]
Open Chat	^⌘I	Ctrl+Alt+I
Inline chat	⌘I	Ctrl+I
Pick agent / model	Chat input pickers	Chat input pickers

Sheet 29

Quick Reference — Settings, CLI & Files

top settings keys · code CLI flags · .vscode/ files · variables reference

SETTINGS WORTH KNOWING (04, 14)

KEY	VALUE
files.autoSave	"onFocusChange"
editor.formatOnSave	true
editor.defaultFormatter	"publisher.ext"
editor.codeActionsOnSave	{ "source.fixAll": "explicit" }
editor.tabSize	2
editor.rulers	[80]
editor.wordWrap	"on"
editor.stickyScroll.enabled	true
workbench.editor.enablePreview	false
files.trimTrailingWhitespace	true
search.exclude	{ "**/dist": true }
git.autofetch	true

CODE CLI (01, 23)

COMMAND	DOES
code .	Open folder
code -n / -r	New / reuse window
code -g f:l:c	Go to line and column
code -d a b	Diff two files
code --wait	Wait for close (Git editor)
--install-extension id	Install extension
--list-extensions	List installed
--disable-extensions	Start without extensions
--profile "Name"	Open with a profile

.VSCODE/ FILES (17, 18, 24, 27)

FILE	HOLDS
settings.json	Workspace settings
launch.json	Debug configurations
tasks.json	Tasks
extensions.json	Recommended extensions
mcp.json	MCP servers
*.code-snippets	Project snippets
.devcontainer/devcontainer.json	Dev Container (25)
*.code-workspace	Multi-root workspace

VARIABLES (17, 18)

VARIABLE	VALUE
${workspaceFolder}	Workspace root
${file}	Current file
${fileBasename}	File name
${fileDirname}	File's folder
${relativeFile}	Path from root
${lineNumber}	Cursor line
${env:NAME}	Environment variable
${input:id}	Prompt from inputs
${command:id}	Result of a command

SETTING SCOPES (04, 05, 24)

SCOPE	WHERE
User	User/settings.json
Profile	Per profile
Workspace	.vscode/settings.json or .code-workspace
Folder	Root's .vscode/ in multi-root
Language	"[python]": { … }

LAUNCH.JSON FIELDS (17)

FIELD	SETS
type	node · debugpy · java · coreclr · go · chrome
request	launch / attach
program	Entry file
args	CLI arguments
cwd	Working folder
env / envFile	Environment
preLaunchTask	Task to run first
console	integratedTerminal

TASKS.JSON FIELDS (18)

FIELD	SETS
label	Name in the picker
type	shell / process / npm
command / args	What runs
group	build / test (+ isDefault)
problemMatcher	$tsc · $eslint-stylish · $gcc
dependsOn	Run other tasks first
isBackground	Watcher task
runOptions.runOn	folderOpen

AI FILES (26, 27)

FILE	HOLDS
.github/copilot-instructions.md	Rules for every request
.github/instructions/*.instructions.md	Rules by applyTo glob
.github/prompts/*.prompt.md	/ prompt commands
.github/agents/*.agent.md	Custom agents
.vscode/mcp.json	MCP servers
AGENTS.md	Rules shared with other agents